

Desarrollo de software guiado por Inteligencia Artificial con prioridad en la calidad

Diseño e implementación de «Reservas Chanantes», una plataforma SaaS *multi-tenant* de reserva de citas en línea

Trabajo de Fin de Máster

Junio de 2026

Resumen

Este Trabajo de Fin de Máster aborda el diseño, la implementación y el despliegue de **Reservas Chanantes**, una plataforma **SaaS *multi-tenant*** de reserva de citas en línea para pequeños negocios de servicios, con un modelo de negocio **B2B2C** sustentado en pagos divididos mediante Stripe Connect. Más allá del producto, el trabajo constituye un **caso de estudio** sobre la viabilidad del **desarrollo de software guiado por Inteligencia Artificial cuando se prioriza la máxima calidad de ingeniería**. Para ello, el sistema se construye aplicando de forma rigurosa **Arquitectura Limpia, diseño dirigido por el dominio (DDD) y desarrollo guiado por pruebas (TDD)**, sobre una pila de TypeScript estricto, Next.js 16 (App Router) y PostgreSQL gestionado por Supabase. Se documentan las decisiones técnicamente no triviales del dominio —el cálculo de disponibilidad, la prevención de reservas solapadas mediante una restricción de exclusión de PostgreSQL, la gestión sensible a la zona horaria y la idempotencia de las notificaciones— y se valida el resultado con una batería de **240 pruebas automatizadas** concentrada en las capas de dominio y aplicación. El trabajo concluye que el desarrollo asistido por IA, sometido a una disciplina de ingeniería explícita, es una vía viable para producir software no trivial y de calidad de producción, y documenta con transparencia sus limitaciones y líneas de evolución.

Palabras clave: SaaS *multi-tenant*, Arquitectura Limpia, *Domain-Driven Design*, *Test-Driven Development*, Next.js, PostgreSQL, Stripe Connect, desarrollo asistido por IA, calidad del software.

Abstract

This Master's Thesis addresses the design, implementation and deployment of **Reservas Chanantes**, a **multi-tenant SaaS** online appointment-booking platform for small service businesses, built on a **B2B2C** business model supported by split payments through Stripe Connect. Beyond the product itself, the work is a **case study** on the feasibility of **AI-guided software development when maximum engineering quality is prioritised**. To this end, the system is built through a rigorous application of **Clean Architecture, Domain-Driven Design (DDD) and Test-Driven Development (TDD)**, on a stack of strict TypeScript, Next.js 16 (App Router) and PostgreSQL managed by Supabase. The non-trivial domain decisions are documented —availability computation, prevention of overlapping bookings via a PostgreSQL exclusion constraint, timezone-aware time handling and notification idempotency— and the result is validated with a suite of **240 automated tests** concentrated in the domain and application layers. The work concludes that AI-assisted development, subject to an explicit engineering discipline, is a viable path to produce non-trivial, production-quality software, and transparently documents its limitations and avenues for future evolution.

Keywords: multi-tenant SaaS, Clean Architecture, Domain-Driven Design, Test-Driven Development, Next.js, PostgreSQL, Stripe Connect, AI-assisted development, software quality.

Índice general

#	Capítulo	Contenido
1	Introducción y Objetivos	Contexto, justificación funcional y metodológica, objetivos, alcance y estructura
2	Estado del Arte y Tecnologías	Marco conceptual y justificación razonada de la pila tecnológica frente a alternativas
3	Análisis de Requisitos y Casos de Uso	20 requisitos funcionales trazables, requisitos no funcionales y casos de uso representativos
4	Diseño y Arquitectura	Las cuatro capas de la Arquitectura Limpia, modelo de persistencia y gestión del estado
5	Implementación	Disponibilidad, integridad ante concurrencia, zona horaria, Stripe Connect, <i>claim-and-release</i> , RLS
6	Estrategia de Pruebas y Control de Calidad	Pirámide de 240 pruebas, TDD, análisis estático y limitaciones del marco de calidad
7	Conclusiones y Líneas Futuras	Grado de cumplimiento de objetivos, valoración metodológica y trabajo futuro priorizado
—	Bibliografía	Referencias académicas (IEEE) y documentación técnica consultada
—	Anexos	A. Diccionario de datos · B. Políticas RLS · C. Historial de migraciones · D. Variables de entorno · E. Evaluación OWASP Top 10 · F. Estrategia de pruebas y cobertura

Detalle de secciones

- **Cap. 1 · Introducción y Objetivos** — 1.1 Contexto · 1.2 Justificación · 1.3 Objetivos · 1.4 Alcance · 1.5 Estructura del documento
- **Cap. 2 · Estado del Arte y Tecnologías** — 2.1 Panorama de soluciones · 2.2 Marco conceptual · 2.3 Criterios de selección · 2.4 TypeScript · 2.5 Next.js · 2.6 Supabase · 2.7 Stripe Connect · 2.8 Resend · 2.9 Tailwind · 2.10 Vitest · 2.11 Síntesis
- **Cap. 3 · Requisitos y Casos de Uso** — 3.1 Introducción · 3.2 Actores · 3.3 Requisitos funcionales · 3.4 Requisitos no funcionales · 3.5 Modelado de casos de uso · 3.6 Especificación (CU-05, CU-08, CU-09)
- **Cap. 4 · Diseño y Arquitectura** — 4.1 Visión general · 4.2 Dominio · 4.3 Aplicación · 4.4 Infraestructura · 4.5 Presentación · 4.6 Estado · 4.7 Persistencia · 4.8 Síntesis
- **Cap. 5 · Implementación** — 5.1 Criterio · 5.2 Disponibilidad · 5.3 Concurrencia · 5.4 Zona horaria · 5.5 Stripe Connect · 5.6 *Claim-and-release* · 5.7 RLS · 5.8 i18n · 5.9 Síntesis
- **Cap. 6 · Pruebas y Calidad** — 6.1 Filosofía · 6.2 Marco · 6.3 Distribución · 6.4 Dominio · 6.5 Aplicación · 6.6 Infraestructura · 6.7 Análisis estático y limitaciones · 6.8 Síntesis
- **Cap. 7 · Conclusiones** — 7.1 Cumplimiento de objetivos · 7.2 Valoración metodológica · 7.3 Limitaciones · 7.4 Líneas futuras · 7.5 Conclusión final

Ficha técnica del proyecto

Producto	Reservas Chanantes — SaaS <i>multi-tenant</i> de reserva de citas
Código fuente	booking-saas/ (Next.js 16 · React 19 · TypeScript estricto)
Despliegue	https://reservas-chanantes.vercel.app/ (Vercel, <i>serverless</i>)
Persistencia	PostgreSQL gestionado por Supabase · 5 tablas · 10 migraciones
Pagos	Stripe Connect Express (modelo B2B2C)
Pruebas	240 casos en 32 ficheros (Vitest); cobertura con umbral, CI y E2E (Playwright)
Arquitectura	Clean Architecture: <code>domain</code> → <code>application</code> → <code>infrastructure</code> → <code>presentation</code>

Capítulo 1. Introducción y Objetivos

1.1. Contexto

El presente Trabajo de Fin de Máster documenta el diseño y la implementación de **Reservas Chanantes**, una **plataforma SaaS (Software as a Service) multi-tenant de reserva de citas en línea** orientada a pequeños negocios de servicios con cita previa —peluquerías, centros de fisioterapia, estética y similares—. El sistema permite que un negocio se registre como **inquilino (tenant)**, configure su catálogo de servicios y su horario de apertura, y publique una **URL pública propia** (por ejemplo, `/peluqueria-juan`) desde la que sus clientes finales consultan la disponibilidad en tiempo real, reservan una cita y, opcionalmente, abonan el servicio en línea.

La aplicación responde a una necesidad real del segmento de microempresas y autónomos, que con frecuencia gestionan sus citas por teléfono o mensajería instantánea, sin un sistema que evite solapamientos, calcule la disponibilidad de forma fiable o integre el cobro. Las soluciones comerciales existentes suelen resultar costosas o excesivamente genéricas para este perfil, lo que justifica el desarrollo de una alternativa **ligera, de bajo coste operativo y arquitectónicamente sólida**.

Desde el punto de vista del modelo de negocio, el sistema adopta un esquema **B2B2C (business-to-business-to-consumer)**: la plataforma presta servicio a los negocios (B2B) y estos, a su vez, atienden a sus propios clientes finales (B2C). Esta naturaleza se refleja directamente en la integración de pagos mediante **Stripe Connect**, que habilita que cada negocio cobre a sus clientes a través de la plataforma reteniendo esta una comisión.

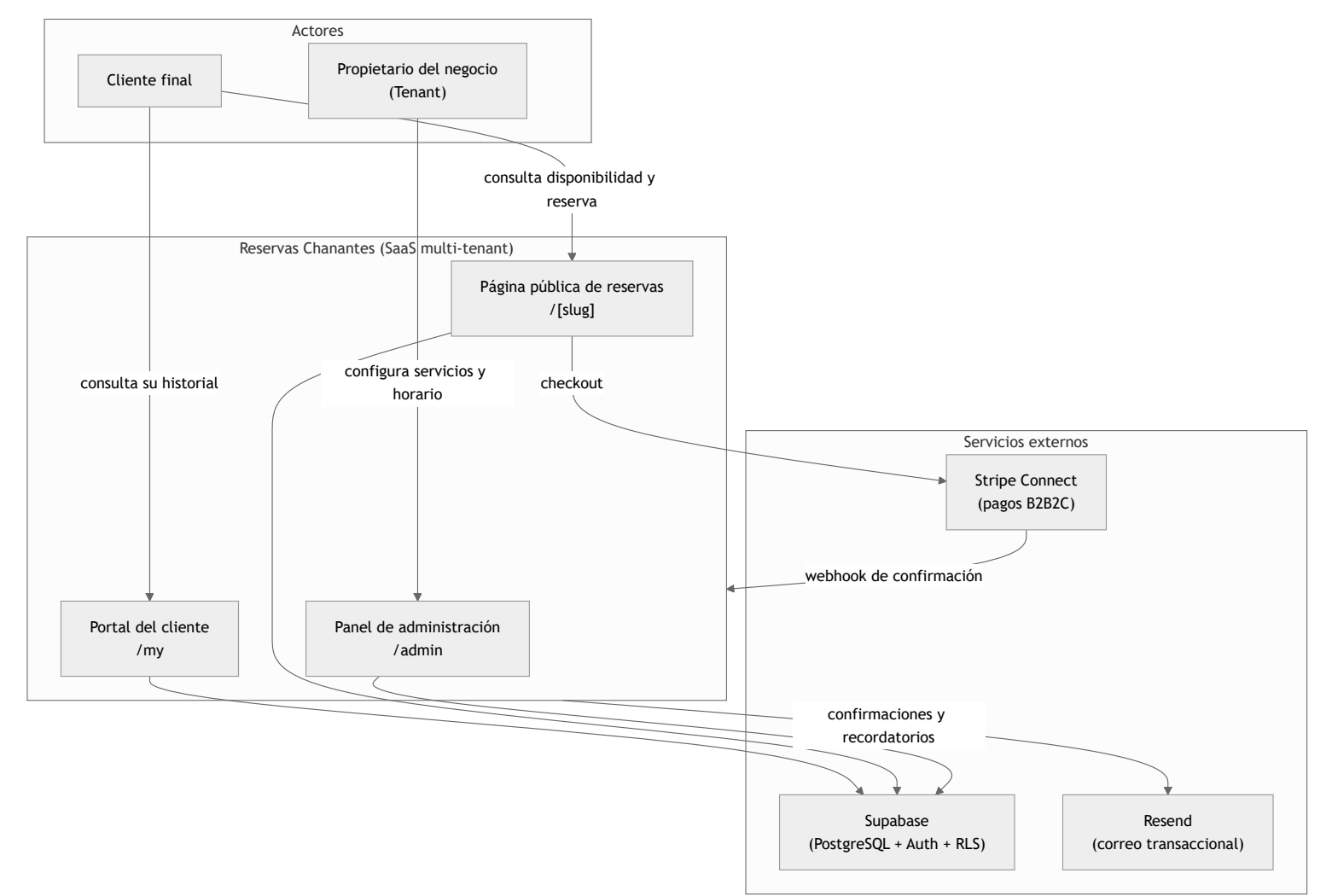


Figura 1.1. Diagrama de contexto del sistema: actores principales, superficies de la aplicación y servicios externos integrados.

1.2. Justificación

La motivación del proyecto es doble y combina una dimensión **funcional** y otra **metodológica**, siendo esta última el verdadero eje vertebrador del Trabajo de Fin de Máster.

- **Justificación funcional.** Se busca cubrir la carencia descrita aportando un producto que resuelva los problemas técnicamente no triviales del dominio: el **cálculo de disponibilidad** a partir del horario de apertura menos las reservas ya ocupadas, la **prevención de reservas solapadas** ante accesos concurrentes, la **gestión horaria sensible a la zona horaria** del negocio y la **integración de pagos** en un contexto multi-negocio.
- **Justificación metodológica.** Tal como se recoge en el documento de diseño original del proyecto, el objetivo fundamental es **demostrar la viabilidad del desarrollo de software guiado por Inteligencia Artificial priorizando la máxima calidad** (docs/plans/2026-02-20-booking-saas-design.md). El proyecto se concibe, por tanto, no solo como un producto, sino como un **caso de estudio** sobre la aplicación rigurosa de principios de ingeniería del software —**Arquitectura Limpia (Clean Architecture)**, **diseño dirigido por el dominio (Domain-Driven Design)**, **desarrollo guiado por pruebas (Test-Driven Development)** y principios **SOLID**— en un flujo de trabajo asistido por IA. La trazabilidad de este proceso queda documentada en los artefactos del repositorio (docs/plans/ , docs/reviews/ y docs/plans/2026-02-20-mvp-deviations.md).

1.3. Objetivos

1.3.1. Objetivo general

Diseñar, implementar y desplegar una plataforma SaaS multi-tenant de reservas en línea de calidad de producción, que sirva simultáneamente como **validación empírica de un proceso de desarrollo asistido por IA centrado en la calidad del software**, evidenciada mediante una arquitectura desacoplada y una cobertura de pruebas sistemática.

1.3.2. Objetivos específicos (técnicos)

Se identifican los siguientes objetivos específicos, todos ellos verificables sobre el código del repositorio:

1. **Aplicar Arquitectura Limpia** con separación estricta de capas (domain , application , infrastructure , presentation), manteniendo el dominio libre de dependencias de *framework*.
2. **Modelar el dominio** mediante entidades, objetos de valor (*value objects*) y servicios de dominio puros que encapsulen las reglas de negocio (cálculo de disponibilidad, políticas de antelación, validaciones).
3. **Garantizar la integridad de las reservas** frente a la concurrencia, llevando la prevención de solapamientos a la capa autoritativa de la base de datos.
4. **Implementar la multi-tenancy** con aislamiento de datos mediante *Row Level Security* (RLS) de PostgreSQL.
5. **Integrar pagos B2B2C** con Stripe Connect, incluyendo el alta de cuentas conectadas y la confirmación de reservas mediante *webhooks*.
6. **Ofrecer una experiencia internacionalizada** (es-ES / en-US) tanto en el panel de administración como en las comunicaciones por correo electrónico.
7. **Desplegar el sistema** en un entorno *serverless* de ejecución continua.

1.3.3. Objetivos metodológicos (de calidad)

1. **Adoptar TDD** como práctica de desarrollo en las capas de dominio y aplicación, sustentando el código en una batería de pruebas automatizadas.
2. **Mantener la trazabilidad del proceso** asistido por IA mediante planes de implementación, documentos de revisión y registro de desviaciones.
3. **Documentar de forma honesta las limitaciones** del sistema, distinguiendo lo implementado de lo planificado como trabajo futuro.

1.4. Alcance

El alcance del Trabajo de Fin de Máster comprende el **producto mínimo viable funcional** del sistema: registro y autenticación de negocios, gestión de servicios y horarios, página pública de reservas con cálculo de disponibilidad, flujo de pago

con Stripe, portal del cliente final y sistema de notificaciones por correo. Quedan **explícitamente fuera del alcance** de la presente entrega, y se abordan como líneas futuras (Capítulo 7), el endurecimiento de determinadas políticas de seguridad y las pruebas de integración contra una base de datos real, según se detalla en el análisis de brechas del proyecto.

1.5. Estructura del documento

La memoria se organiza en siete capítulos. Tras esta **introducción** (Capítulo 1), el **Capítulo 2** justifica la pila tecnológica frente a sus alternativas. El **Capítulo 3** formaliza los requisitos y casos de uso. El **Capítulo 4** detalla el diseño y la arquitectura del sistema. El **Capítulo 5** profundiza en los aspectos más relevantes de la implementación. El **Capítulo 6** describe la estrategia de pruebas y control de calidad. Finalmente, el **Capítulo 7** valora el grado de cumplimiento de los objetivos y propone líneas de evolución futura.

Capítulo 2. Estado del Arte y Tecnologías

2.1. Panorama de las soluciones de reserva en línea

El mercado de la reserva de citas en línea está ocupado por plataformas comerciales propietarias de tipo SaaS —entre las más extendidas, Calendly, Booksy o Treatwell— que ofrecen una funcionalidad amplia a cambio de cuotas periódicas y de un grado de personalización limitado. Para el segmento objetivo de este trabajo (microempresas y profesionales autónomos), dichas soluciones presentan dos inconvenientes recurrentes: un **coste recurrente** difícil de sostener y un modelo de explotación cerrado en el que el negocio no controla plenamente ni sus datos ni su identidad digital.

Frente a este panorama, el sistema desarrollado se plantea como una alternativa **autoalojable y de bajo coste operativo**, sustentada en servicios gestionados con planes de entrada gratuitos. La aportación diferencial del proyecto no reside en el volumen de funcionalidades, sino en el rigor arquitectónico y en la dimensión metodológica expuestos en el Capítulo 1.

2.2. Marco conceptual

La fundamentación teórica del proyecto se apoya en un conjunto de principios consolidados de la ingeniería del software, que constituyen el verdadero estado del arte sobre el que se construye la solución:

- **Arquitectura Limpia (*Clean Architecture*)**: propone organizar el software en capas concéntricas en las que las dependencias apuntan siempre hacia el dominio, de modo que las reglas de negocio permanezcan independientes de los detalles de entrega y de infraestructura [1].
- **Diseño dirigido por el dominio (*Domain-Driven Design*)**: aporta el vocabulario de modelado —entidades, objetos de valor y servicios de dominio— empleado en la capa central del sistema [2].
- **Arquitectura hexagonal (*Ports and Adapters*)**: formaliza el desacoplamiento entre la lógica de aplicación y sus dependencias externas mediante puertos (interfaces) y adaptadores (implementaciones) [3].
- **Patrón Repositorio**: media entre el dominio y la capa de persistencia ofreciendo una interfaz de tipo colección para el acceso a los objetos de dominio [4].
- **Desarrollo guiado por pruebas (*Test-Driven Development*)**: establece el ciclo de escritura de pruebas previas a la implementación que vertebra el desarrollo de las capas de dominio y aplicación [5].
- **Multi-tenancy en aplicaciones SaaS**: define el alojamiento de múltiples clientes (*tenants*) sobre una única instancia de aplicación, junto con las preocupaciones arquitectónicas asociadas —aislamiento de datos, personalización y rendimiento— [6].

Estos principios no son meramente declarativos: su materialización es verificable en la estructura del repositorio y se detalla en el Capítulo 4.

2.3. Criterios de selección tecnológica

La elección de la pila se rige por cuatro criterios derivados de los objetivos del proyecto:

1. **Coherencia con la Arquitectura Limpia**: el *framework* no debe imponer acoplamientos que penetren en el dominio.
2. **Verificabilidad**: las herramientas deben favorecer un ciclo de TDD ágil.
3. **Seguridad de tipos**: se prioriza la detección de errores en tiempo de compilación.
4. **Coste y operación**: se favorecen modelos *serverless* y servicios gestionados que minimicen la administración de infraestructura.

2.4. Lenguaje de programación: TypeScript en modo estricto

Se ha optado por **TypeScript** (`typescript ^5`) frente a JavaScript como lenguaje base. El proyecto activa el **modo estricto** del compilador (`"strict": true` en `tsconfig.json`), que habilita de forma conjunta un grupo de comprobaciones —entre ellas `strictNullChecks` y `noImplicitAny` —, reforzando el criterio de seguridad de tipos. Esta garantía resulta especialmente valiosa en la capa de dominio, donde los objetos de valor (por ejemplo, `Money` o `TimeRange`) dependen de invariantes que el sistema de tipos

contribuye a preservar. La configuración define además un **alias de módulos** (`"@/*": [". /src/*"]`) que habilita importaciones absolutas y clarifica las dependencias entre capas.

2.5. Framework de aplicación: Next.js 16 (App Router)

El núcleo de la aplicación se construye sobre **Next.js 16.1.6** con **React 19.2**, empleando el paradigma **App Router**. La elección se sustenta en tres capacidades que el repositorio utiliza de forma directa:

- **React Server Components (RSC)**: componentes que se renderizan en el servidor, en un entorno separado del cliente, lo que reduce el JavaScript enviado al navegador y permite el acceso a la infraestructura sin exponer secretos [7]. Los RSC alcanzaron estabilidad en React 19; según la documentación oficial de React, el App Router de Next.js constituye la implementación de RSC más madura y lista para producción disponible en la actualidad [7].
- **Server Actions**: la mutación de datos se canaliza a través de funciones de servidor —el repositorio contiene **diez** ficheros `actions.ts` en `src/app/**` — que actúan como **controladores** en términos de la Arquitectura Limpia, delegando en los casos de uso de la capa de aplicación.
- **Interceptación de peticiones**: la gestión transversal de la sesión y la protección de rutas se implementa en `src/proxy.ts`. Conviene precisar que en Next.js 16 el fichero `proxy` **sustituye al anterior convenio middleware** y se ejecuta, por defecto, en el entorno de ejecución **Node.js**. Este interceptor refresca la sesión y protege las rutas `/admin` (salvo `login` y `register`) y `/my` (salvo `login`).

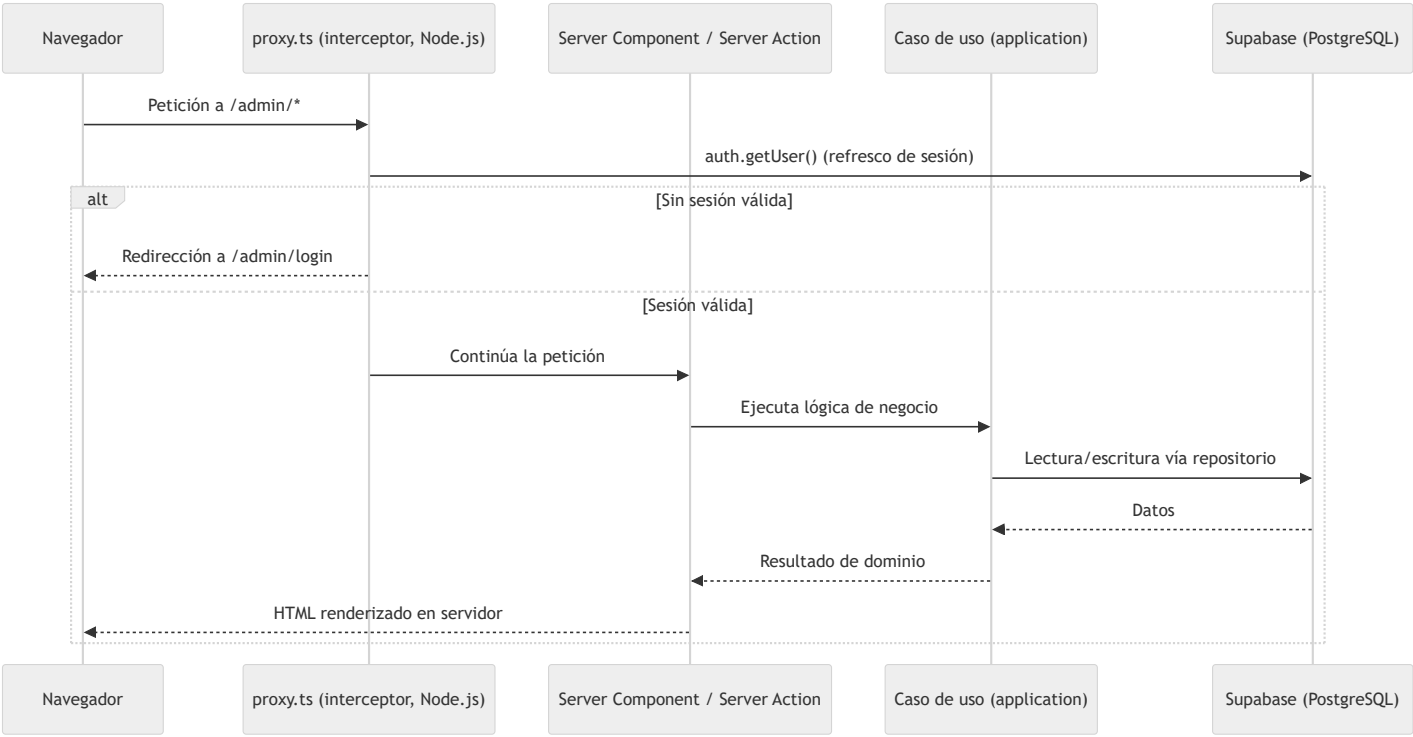


Figura 2.1. Modelo de ejecución SSR con interceptación de sesión (`src/proxy.ts`) y delegación en los casos de uso.

Respecto a las alternativas, la siguiente tabla resume la comparación frente a los principales candidatos:

Criterio	Next.js (App Router)	SPA (React + Express)	Remix	Nuxt
Ecosistema base	React	React	React	Vue (no React)
React Server Components	Sí, estables	No	Parcial / en desarrollo	No aplica
Tipado extremo a extremo	Nativo	Requiere duplicación	Nativo	Nativo (Vue)
Despliegue <i>serverless</i>	Integrado	Manual	Integrado	Integrado

Tabla 2.1. Comparativa del framework de aplicación frente a alternativas.

Una arquitectura tradicional de **SPA con API REST en Express** habría exigido duplicar la definición de tipos entre cliente y servidor y gestionar manualmente la serialización y la autenticación. **Remix** ofrece capacidades SSR comparables sobre React, si bien su soporte de RSC se encontraba en desarrollo en el momento de la elección. **Nuxt**, por su parte, pertenece al ecosistema **Vue**, lo que lo descartaba para un proyecto basado en React. Next.js se seleccionó por integrar de forma estable los RSC, los *Server Actions* y el despliegue *serverless*.

2.6. Persistencia y autenticación: Supabase

La persistencia y la autenticación se delegan en **Supabase** (`@supabase/supabase-js ^2.9.7` , `@supabase/ssr ^0.8`), una plataforma gestionada que expone **PostgreSQL** como base de datos relacional junto con servicios de autenticación y de seguridad a nivel de fila (*Row Level Security*, RLS). La integración se materializa en tres clientes diferenciados:

- **Cliente de navegador** (`createSupabaseBrowser` , en `client.ts`), para componentes de cliente.
- **Cliente de servidor** (`createSupabaseServer` , en `server.ts`), que sincroniza la sesión mediante *cookies* a través de `next/headers` .
- **Cliente administrativo** (`createSupabaseAdmin` , en `admin-client.ts`), con privilegios elevados para operaciones de confianza, como la confirmación de reservas desde los *webhooks*.

El factor decisivo es el motor relacional: características avanzadas empleadas en el Capítulo 5 —como los rangos (`int4range`) y las **restricciones de exclusión** con `btree_gist` para impedir solapamientos— solo están disponibles en un sistema como PostgreSQL.

Criterio	Supabase (PostgreSQL)	Firebase (Firestore)	Backend propio
Modelo de datos	Relacional (SQL)	Documental (NoSQL)	Variable
Integridad referencial	Nativa (claves foráneas, <i>checks</i>)	Limitada	Manual
Restricciones de exclusión / rangos	Sí (<code>EXCLUDE</code> , <code>btree_gist</code>)	No disponible	Compleja
Seguridad a nivel de fila	RLS declarativa	Reglas propietarias	A implementar
Operación	Gestionada	Gestionada	Autogestionada

Tabla 2.2. Comparativa de la capa de persistencia frente a alternativas.

Se descartó **Firebase** porque su modelo documental no permite expresar las **restricciones de exclusión relacionales** que el dominio exige para prevenir solapamientos de reservas, además de dificultar la integridad referencial entre `tenants` , `services` y `bookings` . Un **backend propio** habría incrementado el coste operativo sin aportar ventajas funcionales dentro del alcance definido.

Conviene matizar, en aras del rigor, que el uso de RLS no equivale por sí solo a un aislamiento estricto entre *tenants*: las políticas vigentes en el esquema inicial son deliberadamente permisivas en las tablas `bookings` y `customers` (acceso público de lectura e inserción), una limitación que se analiza en el Capítulo 5 y se propone subsanar en el Capítulo 7.

2.7. Pasarela de pagos: Stripe y Stripe Connect

El cobro se implementa con **Stripe** (`stripe ^20.3`) a través de su SDK oficial, instanciado de forma perezosa como *singleton* en `src/infrastructure/stripe/client.ts` . La naturaleza **B2B2C** del producto motiva el uso específico de **Stripe Connect**, que permite a la plataforma orquestar pagos en nombre de cuentas de terceros (los negocios) reteniendo una comisión. La verificación de las notificaciones se realiza mediante la firma de los *webhooks* en `src/app/api/webhooks/**` .

Criterio	Stripe Connect	Adyen for Platforms	PayPal	Redsys
Modelo de <i>marketplace</i> (cuentas conectadas)	Nativo	Nativo	Limitado	No nativo
Orientación	Pymes y plataformas	Gran empresa	Generalista	Banca (España)
Experiencia de desarrollo / API	Amplia y documentada	Orientada a integración empresarial	Heterogénea	Centrada en TPV/banca
Barrera de entrada para pequeños negocios	Baja	Alta	Media	Media-alta

Tabla 2.3. Comparativa de pasarelas de pago para un modelo B2B2C.

Adyen ofrece un modelo de plataforma técnicamente equiparable, si bien está orientado a clientes de gran tamaño. **PayPal** dispone de soluciones de *marketplace*, aunque su modelo de pagos divididos resulta menos homogéneo para integraciones de pequeña escala. **Redsys**, ampliamente implantado en España, se orienta al procesamiento de tarjetas de un único comercio y no contempla de forma nativa el reparto de pagos entre múltiples partes. Stripe Connect se seleccionó por combinar un modelo de *marketplace* nativo con una baja barrera de entrada y una experiencia de desarrollo adecuada al alcance del proyecto.

2.8. Correo transaccional: Resend

Las comunicaciones (confirmaciones y recordatorios de reserva) se envían mediante **Resend** (`resend ^6.9`). La capa de infraestructura (`src/infrastructure/resend/`) encapsula la composición de plantillas, su traducción (es-ES / en-US) y el formateo de fechas, manteniendo el envío detrás de un puerto de la capa de aplicación (`notification-service.ts`). Esta abstracción permite sustituir el proveedor (por ejemplo, por SendGrid o Amazon SES) sin alterar la lógica de negocio. Cabe señalar que la implementación actual registra los fallos de envío sin propagarlos al flujo principal, decisión cuyo endurecimiento se contempla entre las líneas futuras.

2.9. Interfaz de usuario: Tailwind CSS 4

La capa de presentación utiliza **Tailwind CSS 4**, un *framework* de utilidades CSS integrado de forma nativa con el motor de PostCSS de Next.js (`@tailwindcss/postcss`). Su modelo *utility-first* favorece la consistencia visual y evita la proliferación de hojas de estilo ad hoc.

2.10. Pruebas: Vitest

El marco de pruebas es **Vitest 4** (`vitest ^4.0.18`), configurado con `vite-tsconfig-paths` y `@vitejs/plugin-react` . Se seleccionó frente a **Jest** atendiendo a dos criterios objetivos del proyecto: su compatibilidad nativa con módulos ESM y TypeScript —que evita la capa de transpilación adicional que Jest requiere— y su integración directa con la cadena de herramientas de Vite empleada en el resto del entorno. Esta elección sostiene el ciclo de TDD descrito en el Capítulo 6.

2.11. Síntesis de la pila tecnológica

Capa	Tecnología	Versión	Justificación principal
Lenguaje	TypeScript (strict)	<code>^5</code>	Seguridad de tipos en el dominio
Framework	Next.js (App Router)	<code>16.1.6</code>	RSC, <i>Server Actions</i> y SSR
Vista	React	<code>19.2.3</code>	Componentes de servidor y cliente
Estilos	Tailwind CSS	<code>^4</code>	Diseño <i>utility-first</i> consistente
Datos / Auth	Supabase (PostgreSQL)	<code>ssr ^0.8</code> / <code>js ^2.97</code>	RLS y restricciones de concurrencia
Pagos	Stripe Connect	<code>^20.3</code>	Modelo de <i>marketplace</i> B2B2C
Correo	Resend	<code>^6.9</code>	API transaccional desacoplable
Pruebas	Vitest	<code>^4.0.18</code> 9 / 40	TDD nativo en ESM/TS

En conjunto, la pila satisface los cuatro criterios de la sección 2.3: habilita una arquitectura desacoplada, favorece la verificabilidad, maximiza la seguridad de tipos y se apoya en servicios gestionados de bajo coste operativo.

Referencias del capítulo

- [1] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.
- [2] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.
- [3] A. Cockburn, "Hexagonal Architecture (Ports and Adapters)," 2005. [En línea]. Disponible: <https://alistair.cockburn.us/hexagonal-architecture/>
- [4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.
- [5] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2002.
- [6] R. Krebs, C. Momm y S. Kounev, "Architectural Concerns in Multi-Tenant SaaS Applications," en *Proc. 2nd Int. Conf. on Cloud Computing and Services Science (CLOSER 2012)*, SciTePress, 2012, pp. 426–431.
- [7] React Team, "Server Components," *React Documentation*. [En línea]. Disponible: <https://react.dev/reference/rsc/server-components>

Las referencias citadas ([1] – [7]) se recogen también en la **Bibliografía** consolidada de la memoria.

Capítulo 3. Análisis de Requisitos y Casos de Uso

3.1. Introducción

Este capítulo formaliza los requisitos del sistema y modela su comportamiento mediante casos de uso. El análisis se ha realizado por **ingeniería inversa sobre el código existente**, de modo que cada requisito es trazable hasta su implementación concreta en el repositorio. Se distinguen los **requisitos funcionales** (qué debe hacer el sistema), los **requisitos no funcionales** (con qué cualidades) y la especificación de los casos de uso más representativos.

3.2. Actores del sistema

Se identifican cuatro actores, dos humanos y dos no humanos:

Actor	Tipo	Descripción
Propietario del negocio	Humano	Usuario registrado (<i>tenant</i>) que configura el negocio, sus servicios y su horario, y gestiona las reservas recibidas. Opera en <code>/admin</code> .
Cliente final	Humano	Persona que reserva una cita. Puede actuar de forma anónima (página pública <code>/[slug]</code>) o autenticada (portal <code>/my</code>).
Sistema de tareas programadas	No humano	Proceso automático (<i>cron</i>) que dispara el envío de recordatorios de reserva (<code>/api/cron/send-reminders</code>).
Pasarela de pago (Stripe)	No humano	Sistema externo que confirma los pagos y el estado de las cuentas conectadas mediante <i>webhooks</i> (<code>/api/webhooks/**</code>).

El proveedor de identidad Google no se modela como actor independiente, sino como mecanismo de autenticación del actor humano a través de Supabase Auth.

3.3. Requisitos funcionales

Los requisitos funcionales se agrupan por actor. La columna *Implementación* aporta la trazabilidad exigida (regla de verificación frente al código).

3.3.1. Propietario del negocio (RF-A)

ID	Requisito	Implementación
RF-A1	Registrar un negocio y crear su cuenta de propietario	<code>admin/register/actions.ts</code> (<code>register</code>)
RF-A2	Autenticarse mediante correo/contraseña o Google OAuth; el registro por correo exige confirmación de email	<code>admin/login/actions.ts</code> (<code>login</code>); <code>signInWithOAuth</code> ; <code>api/auth/callback</code> (OAuth) y <code>api/auth/confirm</code> (confirmación por enlace)
RF-A3	Recuperar la contraseña olvidada	<code>admin/login/actions.ts</code> (<code>resetPassword</code>) y página <code>admin/reset-password/</code> (<code>updatePassword</code>)
RF-A4	Crear, editar y eliminar servicios (nombre, duración, precio, estado)	<code>admin/(dashboard)/services/actions.ts</code> (<code>saveService</code> , <code>deleteService</code>)
RF-A5	Definir el horario semanal de apertura (varios rangos por día)	<code>admin/(dashboard)/schedule/actions.ts</code> (<code>saveSchedule</code>)
RF-A6	Consultar y cancelar reservas, con verificación de propiedad	<code>admin/(dashboard)/bookings/actions.ts</code> (<code>cancelBooking</code>)
RF-A7	Configurar el negocio: zona horaria, política de antelación y perfil	<code>admin/(dashboard)/settings/actions.ts</code> (<code>saveSettings</code>)
RF-A8	Conectar una cuenta de cobro para recibir pagos	<code>application/use-cases/connect-stripe-account.ts</code> ; <code>api/stripe/connect</code>

3.3.2. Cliente final (RF-C)

ID	Requisito	Implementación
RF-C1	Consultar la disponibilidad de un negocio en tiempo real	<code>[slug]/actions.ts (getAvailability); application/use-cases/get-availability.ts</code>
RF-C2	Crear una reserva sobre un hueco disponible	<code>[slug]/actions.ts (createBooking); application/use-cases/create-booking.ts</code>
RF-C3	Pagar el servicio en línea	<code>infrastructure/stripe/payment-service.ts (createCheckoutSession);</code> confirmación vía <i>webhook</i>
RF-C4	Registrarse e iniciar sesión como cliente	<code>my/login/actions.ts (customerRegister , customerLogin)</code>
RF-C5	Vincular la cuenta autenticada con su historial de reservas	<code>application/use-cases/link-customer-auth.ts</code>
RF-C6	Consultar su historial de reservas	<code>application/use-cases/get-customer-bookings.ts ; my/(portal)/history</code>
RF-C7	Cancelar una de sus reservas (autoservicio)	<code>my/(portal)/actions.ts (cancelCustomerBooking); application/use-cases/cancel-customer-booking.ts</code>
RF-C8	Actualizar los datos de su perfil	<code>my/(portal)/profile/actions.ts (updateProfile); application/use-cases/update-customer-profile.ts</code>

3.3.3. Sistema e integraciones (RF-S)

ID	Requisito	Implementación
RF-S1	Enviar correos de confirmación tras un pago satisfactorio	<code>infrastructure/resend/send-booking-emails.ts ; webhook stripe-connect</code>
RF-S2	Enviar recordatorios automáticos de reserva, sin duplicados	<code>api/cron/send-reminders/route.ts</code> (patrón <i>claim-and-release</i>)
RF-S3	Confirmar la reserva al completarse el pago	<code>api/webhooks/stripe-connect/route.ts</code> (<code>checkout.session.completed</code>)
RF-S4	Actualizar el estado de la cuenta de cobro del negocio	<code>api/webhooks/stripe/route.ts</code> (<code>account.updated</code>)

El sistema cuenta, por tanto, con **20 requisitos funcionales** (8 del propietario, 8 del cliente y 4 del sistema).

3.4. Requisitos no funcionales

ID	Categoría	Requisito	Sustento en el sistema
RNF-1	Seguridad	Autenticación de usuarios y protección de rutas privadas	Supabase Auth; interceptor <code>src/proxy.ts</code>
RNF-2	Seguridad	Verificación de la autenticidad de las notificaciones de pago	Firma de <i>webhooks</i> (<code>constructEvent</code>)
RNF-3	Integridad	Imposibilidad de reservas solapadas ante concurrencia	Restricción <code>EXCLUDE / btree_gist</code> sobre <code>int4range(start_minutes, end_minutes, '[)')</code> (excluyendo reservas canceladas) en PostgreSQL
RNF-4	Internacionalización	Interfaz y correos en es-ES y en-US	<code>infrastructure/i18n/</code> , <code>infrastructure/resend/</code>
RNF-5	Mantenibilidad	Separación estricta de capas y dominio independiente	Arquitectura Limpia (Capítulo 4)
RNF-6	Verificabilidad	Cobertura de pruebas automatizadas en dominio y aplicación	Suite Vitest (Capítulo 6)
RNF-7	Rendimiento	Renderizado en servidor e indexación de consultas frecuentes	RSC; índices sobre <code>tenant_id</code> , <code>slug</code> , <code>(tenant_id, date)</code>
RNF-8	Fiabilidad	Recordatorios idempotentes frente a fallos parciales	Patrón <i>claim-and-release</i> en el <i>cron</i>
RNF-9	Usabilidad	Estados de carga y de error en todas las superficies	Convenciones <code>loading.tsx</code> / <code>error.tsx</code>
RNF-10	Disponibilidad	Despliegue <i>serverless</i> de ejecución continua	Plataforma Vercel

Se hace constar, en coherencia con el Capítulo 2, que el aislamiento de datos entre *tenants* (un requisito de seguridad esperable en un SaaS maduro) está **parcialmente cubierto**: las políticas RLS actuales son permisivas en determinadas tablas. Esta brecha se documenta como línea futura en el Capítulo 7.

3.5. Modelado de casos de uso

La siguiente figura representa el diagrama de casos de uso del sistema, organizado en torno a la frontera de la aplicación y sus actores.

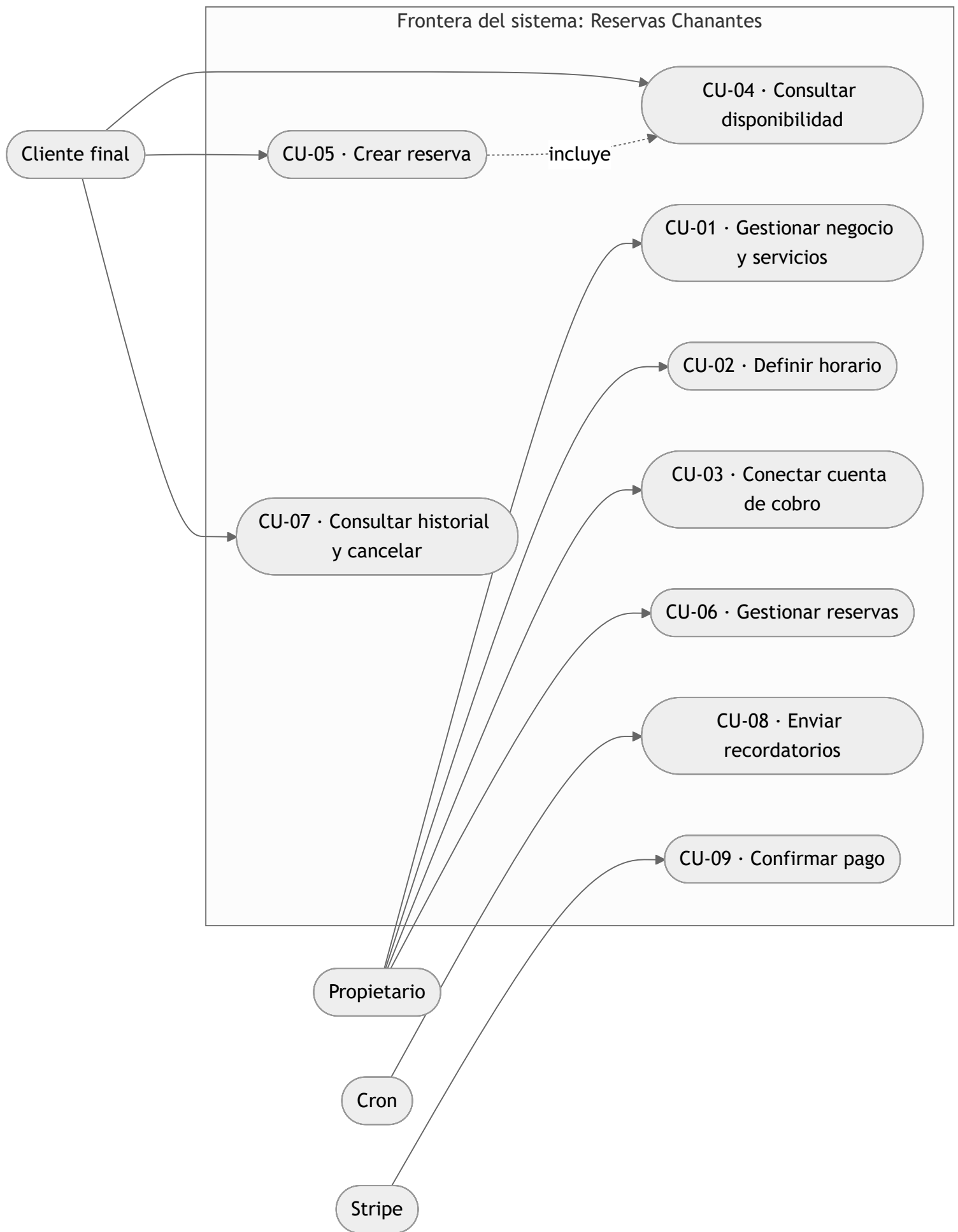


Figura 3.1. Diagrama de casos de uso del sistema (notación aproximada en Mermaid).

3.6. Especificación de casos de uso representativos

Se detallan a continuación los casos de uso de mayor complejidad, por concentrar la lógica de negocio crítica.

3.6.1. CU-05 — Crear reserva

Campo	Descripción
Actor principal	Cliente final
Precondiciones	El negocio existe y tiene servicios y horario configurados
Disparador	El cliente selecciona un servicio, una fecha y una hora
Postcondición (éxito)	Se persiste una reserva en estado <code>PENDING</code> y se bloquea el hueco

Flujo principal (según `application/use-cases/create-booking.ts`):

- El sistema verifica que el negocio (*tenant*) existe.
- Se valida la fecha frente a la **política de antelación** del negocio: no puede exceder la antelación máxima ni ser una fecha pasada.
- Para reservas en el día en curso, se valida la **antelación mínima** respecto a la hora local del negocio.
- Se verifica que el servicio existe y pertenece al negocio.
- Se obtiene el horario del día y se calculan los huecos libres (horario de apertura menos reservas existentes).
- Se comprueba que el servicio **cabe** en el hueco seleccionado.
- Se valida el **teléfono** del cliente (mínimo de dígitos); el caso de uso lanza `InvalidPhoneError` si no cumple.
- Se **normaliza y valida el correo electrónico** mediante el objeto de valor `EmailAddress`, que lanza `InvalidEmailError` ante un formato inválido.
- Se localiza o se crea el cliente por correo electrónico (*upsert*).
- Se persiste la reserva en estado `PENDING`.

Excepciones lanzadas por el propio caso de uso (`create-booking.ts`): `TenantNotFoundError`, `BookingTooFarAheadError`, `BookingInPastError`, `BookingTooSoonError`, `ServiceNotFoundError`, `BusinessClosedError`, `ServiceDoesNotFitError`, `InvalidPhoneError`.

Excepciones originadas fuera del caso de uso pero que afloran en este flujo: `InvalidEmailError` (lanzada por el objeto de valor `EmailAddress`) y `SlotTakenError` (lanzada por la restricción `EXCLUDE` de la base de datos a través del repositorio y capturada en la *server action* `[slug]/actions.ts`).

3.6.2. CU-08 — Enviar recordatorios

Campo	Descripción
Actor principal	Sistema de tareas programadas (<i>cron</i>)
Precondiciones	Existen reservas confirmadas próximas sin recordatorio enviado
Postcondición (éxito)	Cada reserva recibe, como máximo, un recordatorio

Flujo principal (según `api/cron/send-reminders/route.ts`): para cada reserva candidata, el sistema **reclama** atómicamente el envío (`claimReminder`, implementado como un *compare-and-swap* mediante `UPDATE ... WHERE reminder_sent_at IS NULL`); si la reclamación tiene éxito, envía el recordatorio; ante un fallo de envío, **libera** la reclamación (`releaseReminder`) para que un ciclo posterior la reintente. Este patrón garantiza el requisito RNF-8 (no duplicación).

3.6.3. CU-09 — Confirmar pago

Campo	Descripción
Actor principal	Pasarela de pago (Stripe)
Precondiciones	Existe una reserva en estado <code>PENDING</code> asociada a la sesión de pago
Postcondición (éxito)	La reserva pasa a <code>CONFIRMED</code> y se envían los correos de confirmación

Flujo principal (según `api/webhooks/stripe-connect/route.ts`): Stripe notifica el evento `checkout.session.completed`; el sistema **verifica la firma** del *webhook*, recupera la reserva referenciada en los metadatos y, solo si sigue en estado `PENDING`, la

confirma y dispara las notificaciones.

4.1. Visión arquitectónica general

El sistema implementa la **Arquitectura Limpia** descrita en el marco conceptual (Capítulo 2), organizando el código en **cuatro capas concéntricas** que respetan una única **regla de dependencias**: el código fuente solo puede depender hacia el interior, nunca hacia el exterior. El dominio, en el centro, no conoce ni el *framework* ni la base de datos; la infraestructura, en el exterior, depende de las abstracciones definidas en las capas internas.

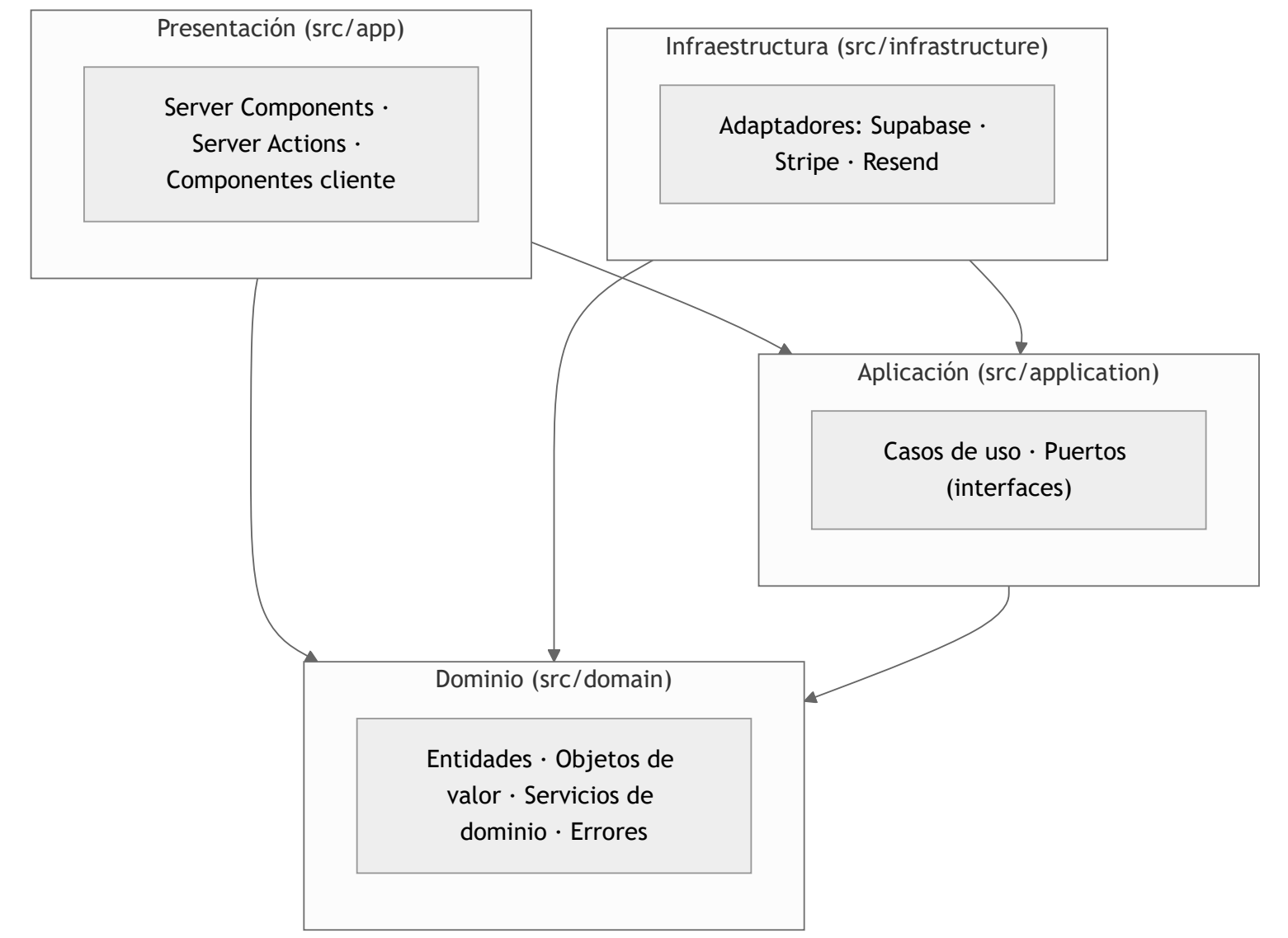


Figura 4.1. Capas de la Arquitectura Limpia y regla de dependencias (las flechas apuntan hacia el dominio).

Esta disposición se corresponde literalmente con la estructura de directorios del repositorio: `src/domain`, `src/application`, `src/infrastructure` y `src/app`.

4.2. Capa de dominio

La capa de dominio (`src/domain`) concentra las reglas de negocio y **no contiene ninguna dependencia de *framework***. Se compone de cuatro elementos:

4.2.1. Entidades

Las entidades modelan los conceptos centrales con identidad propia: `Tenant`, `Service`, `Booking`, `Customer` y `WeeklySchedule`. Se definen como interfaces de propiedades **inmutables** (`readonly`) —salvo `WeeklySchedule`, modelada como clase—. Por ejemplo,

la entidad `Tenant` (`domain/entities/tenant.ts`) agrega no solo datos primitivos, sino también un objeto de valor (`bookingPolicy`), su plan de suscripción (`plan`) y los atributos de integración con la pasarela de pago (`stripeAccountId` , `stripeAccountEnabled`).

4.2.2. Objetos de valor

El directorio `domain/value-objects` contiene seis ficheros. **Cinco** de ellos son **objetos de valor** propiamente dichos, que encapsulan un concepto sin identidad y **protegen sus invariantes en el momento de la construcción**: `TimeRange` , `Money` , `Slug` , `BookingPolicy` y `EmailAddress` . El sexto, `BusinessCategory` , es un **tipo enumerado** (unión de literales) sin validación en constructor, empleado para clasificar el tipo de negocio.

El objeto `TimeRange` (`domain/value-objects/time-range.ts`) es representativo del patrón: su constructor rechaza cualquier rango no válido (inicio negativo, fin superior a 1440 minutos o inicio no anterior al fin) lanzando `InvalidTimeRangeError` , de modo que **es imposible instanciar un rango temporal inconsistente**. Además, expone comportamiento de dominio rico — `overlaps` , `contains` , `subtract` , `equals` — que la capa de servicios reutiliza para calcular la disponibilidad.

```
constructor(start: number, end: number) {
  if (start < 0 || end > 1440 || start >= end) {
    throw new InvalidTimeRangeError(start, end)
  }
  this.start = start
  this.end = end
}
```

Fragmento 4.1. Protección de invariantes en el constructor de `TimeRange` .

4.2.3. Servicios de dominio

Cuando una operación no pertenece naturalmente a una sola entidad, se modela como **servicio de dominio**: `availability-calculator` (resta de reservas, generación de huecos y verificación de ajuste), `tenant-clock` (operaciones de fecha y hora sensibles a la zona horaria del negocio), `locale-resolver` y `plan-limits` (límites y comisión por plan de suscripción).

4.2.4. Errores de dominio

Los errores se modelan como una jerarquía de clases tipadas (`domain/errors/domain-errors.ts`), lo que permite que las capas superiores los distingan y los traduzcan a mensajes para el usuario sin acoplarse a cadenas de texto.

4.3. Capa de aplicación

La capa de aplicación (`src/application`) orquesta los casos de uso y define los **puertos** que la infraestructura debe implementar.

- **Casos de uso**: siete clases que coordinan las entidades y los servicios para satisfacer un requisito (por ejemplo, `CreateBookingUseCase`).
- **Puertos**: ocho puertos, definidos como interfaces (`application/ports`), que abstraen la persistencia y los servicios externos (`BookingRepository` , `TenantRepository` , `PaymentService` , `NotificationService` , `StripeConnectService` , etc.).

La **inyección de dependencias** se realiza por **constructor**: un caso de uso recibe sus colaboradores como interfaces, sin conocer su implementación concreta. Así, `CreateBookingUseCase` recibe cinco repositorios (`tenant` , `service` , `schedule` , `booking` y `customer`) y permanece comprobable de forma aislada mediante dobles de prueba.

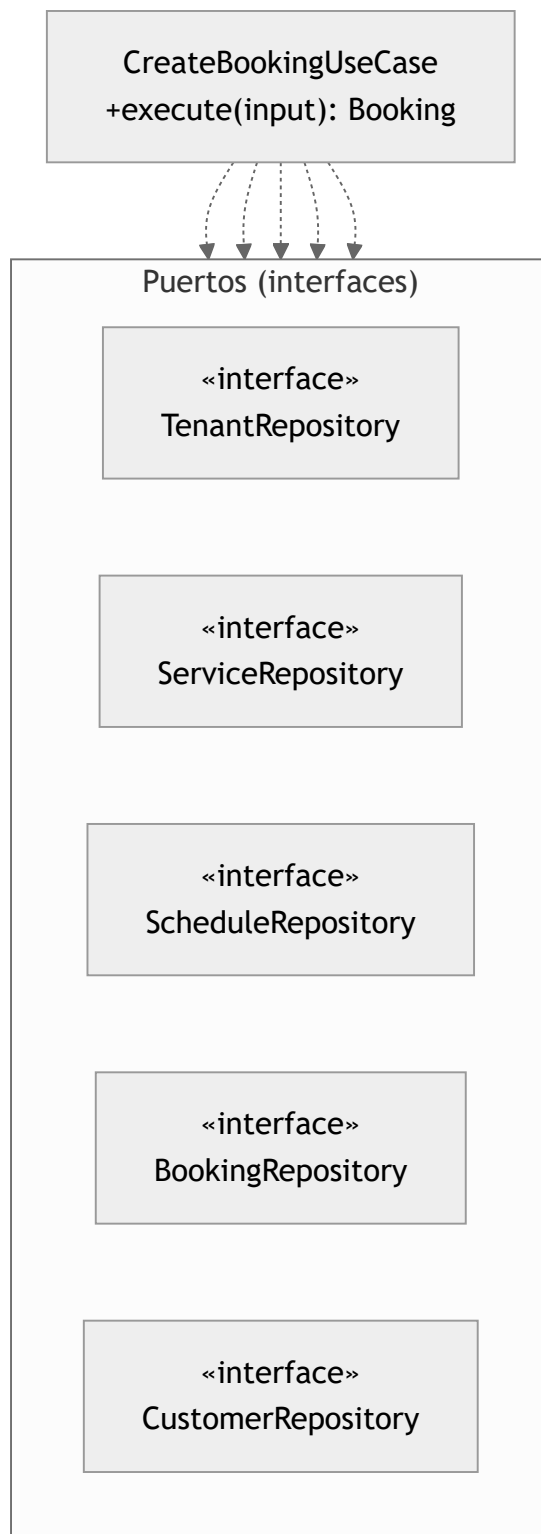


Figura 4.2. El caso de uso depende de los cinco puertos (interfaces), no de implementaciones (inversión de dependencias).

4.4. Capa de infraestructura

La capa de infraestructura (`src/infrastructure`) contiene los **adaptadores** que implementan los puertos de la capa de aplicación:

- **Persistencia:** cinco repositorios sobre Supabase (`SupabaseTenantRepository` , `SupabaseServiceRepository` , `SupabaseScheduleRepository` , `SupabaseBookingRepository` , `SupabaseCustomerRepository`).
- **Pagos:** `StripePaymentService` (`payment-service.ts`) y `StripeConnectServiceImpl` (`stripe-connect-service.ts`).
- **Notificaciones:** `ResendNotificationService` (`resend/`), que implementa el puerto `NotificationService` .

El **ensamblaje de dependencias** se centraliza en una *factory* (`infrastructure/supabase/repositories.ts`), que construye los cinco repositorios a partir de un único cliente de Supabase:

```
export function createRepositories(supabase: SupabaseClient) {
  return {
    tenantRepo: new SupabaseTenantRepository(supabase),
    serviceRepo: new SupabaseServiceRepository(supabase),
    scheduleRepo: new SupabaseScheduleRepository(supabase),
    bookingRepo: new SupabaseBookingRepository(supabase),
    customerRepo: new SupabaseCustomerRepository(supabase),
  }
}
```

Fragmento 4.2. Composición de adaptadores en la factory de repositorios.

4.5. Capa de presentación

La capa de presentación (`src/app`) se construye sobre el App Router de Next.js:

- Los **Server Components** renderizan las páginas en el servidor y solicitan datos a través de los casos de uso.
- Los **Server Actions** (`actions.ts`) actúan como **controladores**: reciben la entrada del usuario, invocan la lógica correspondiente y traducen los errores de dominio a mensajes presentables. Conviene precisar una **asimetría real** del proyecto: los *Server Actions* de los flujos **público** (`[slug]/actions.ts`) y de **portal del cliente** (`my/`) invocan los **casos de uso** de la capa de aplicación, mientras que los del **panel de administración** (`bookings` , `schedule` , `services` , `settings`) acceden **directamente a los repositorios** de infraestructura, sin pasar por la capa de aplicación. Esta desviación parcial de la regla de dependencias en la rama de administración se documenta como deuda arquitectónica en el Capítulo 7.
- Los **componentes cliente** (marcados con `'use client'` , como `google-sign-in-button.tsx` o el selector de huecos) se reservan para la interactividad que requiere estado en el navegador.

4.6. Gestión del estado

A diferencia de una aplicación SPA tradicional, el sistema **no emplea un almacén de estado global** (tipo Redux o Zustand). El estado se gestiona de forma predominantemente **dirigida por el servidor**:

- El estado de los datos reside en la base de datos y se renderiza en cada petición mediante Server Components.
- Tras una mutación (Server Action), la coherencia de la caché se restablece con `revalidatePath` , evitando datos obsoletos.
- El estado de cliente se limita a la interactividad local de formularios y selectores (mediante los *hooks* de React en componentes cliente).
- La única información persistida en el lado del cliente es la **sesión de autenticación**, almacenada en *cookies* y sincronizada por el interceptor `src/proxy.ts` .

4.7. Diseño de la persistencia

La persistencia es **remota**, sobre una base de datos relacional PostgreSQL gestionada por Supabase. El modelo se compone de cinco tablas, evolucionadas a través de diez migraciones versionadas (`supabase/migrations/`).

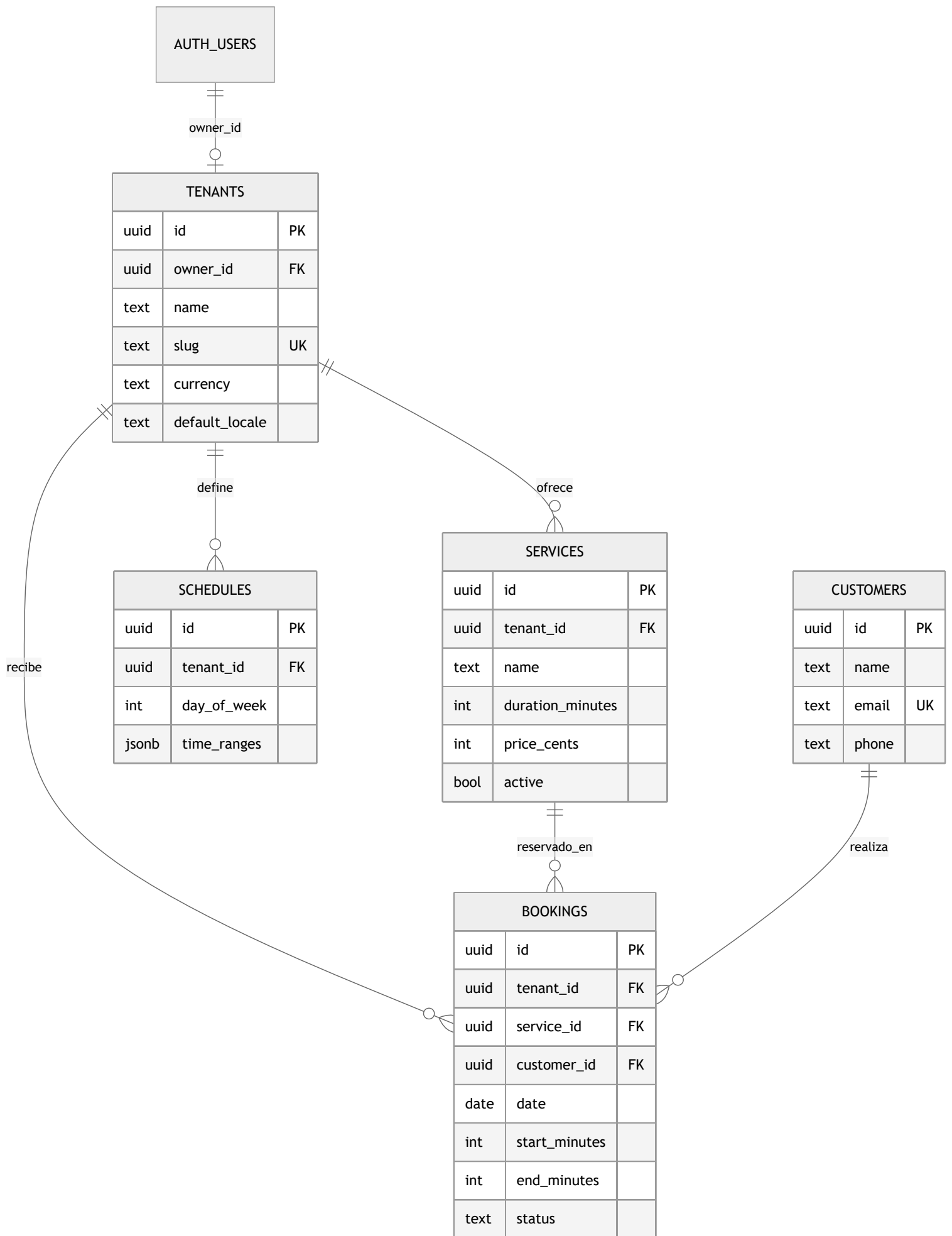


Figura 4.3. Modelo entidad-relación de la persistencia (atributos clave; el diccionario de datos completo se detalla en el Anexo).

La relación entre `auth.users` y `tenants` es de **uno a cero-o-uno**: cada negocio tiene exactamente un propietario, pero no todo usuario autenticado es propietario (los clientes del portal son usuarios sin negocio).

Aspectos destacables del diseño relacional:

- **Integridad referencial** mediante claves foráneas con borrado en cascada (`on delete cascade`).
- **Restricciones de dominio** en la propia base de datos: `check` sobre `duration_minutes > 0` , `start_minutes < end_minutes` y el conjunto de estados de reserva.
- **Horarios flexibles**: el horario de cada día se almacena como `jsonb ( time_ranges )`, permitiendo varios tramos por día.
- **Indexación** de las consultas frecuentes: `tenants(slug)` , `services(tenant_id)` , `schedules(tenant_id)` y `bookings(tenant_id, date)` , entre otros; además, se definen índices únicos sobre `tenants(owner_id)` y `tenants(stripe_account_id)` .
- **Seguridad a nivel de fila (RLS)** habilitada en las cinco tablas; sus políticas se analizan, junto con sus limitaciones actuales, en el Capítulo 5.

Cabe señalar, en aras del rigor, que el atributo `plan` presente en la entidad de dominio `Tenant` **no dispone todavía de columna en el esquema**: el adaptador de persistencia lo resuelve por defecto a `FREE` . En consecuencia, el modelo de monetización (planes FREE/PRO) está diseñado en el dominio, pero su persistencia y aplicación quedan **pendientes**, y se recogen como línea futura en el Capítulo 7.

4.8. Síntesis

El diseño materializa la inversión de dependencias en cada frontera: el dominio define el qué, la aplicación lo orquesta a través de puertos, y la infraestructura aporta el cómo mediante adaptadores intercambiables. Esta separación —verificable en la estructura real del repositorio— es la que habilita la estrategia de pruebas descrita en el Capítulo 6 y sostiene los objetivos de calidad del proyecto, si bien con las desviaciones honestamente señaladas (acceso directo a repositorios en la rama de administración y atributo `plan` no persistido).

Capítulo 5. Implementación

5.1. Criterio de selección de los aspectos

La implementación completa del sistema abarca un volumen considerable de código (aproximadamente 142 ficheros `.ts` / `.tsx` y unas 9.970 líneas). Documentarla exhaustivamente sería ni viable ni útil; este capítulo se concentra, por tanto, en los **aspectos técnicamente no triviales** —aquellos en los que reside la verdadera dificultad del dominio y que sostienen los objetivos de calidad del Capítulo 1—. Se han seleccionado seis: el **cálculo de disponibilidad**, la **integridad de las reservas ante la concurrencia**, la **gestión sensible a la zona horaria**, la **integración de pagos B2B2C**, la **idempotencia de los recordatorios** y la **seguridad multi-tenant** mediante RLS. Cada uno se ilustra con fragmentos extraídos literalmente del repositorio.

5.2. El cálculo de disponibilidad

El núcleo funcional del producto es la respuesta a una pregunta aparentemente simple: *¿qué huecos libres tiene el negocio un día dado?* La disponibilidad no se almacena, sino que se **deriva** de dos fuentes: el horario de apertura del día (uno o varios tramos) menos las reservas ya ocupadas. Esta operación se modela como un **servicio de dominio puro** (`domain/services/availability-calculator.ts`), sin dependencia alguna de la base de datos ni del *framework*.

El algoritmo se apoya en el comportamiento de dominio del objeto de valor `TimeRange` —en particular su método `subtract`, que resta un rango de otro devolviendo de cero a dos rangos resultantes—. La función `subtractBookings` aplica esa resta de forma acumulativa sobre todos los tramos de apertura:

```
export function subtractBookings(
  openingRanges: TimeRange[],
  bookedRanges: TimeRange[]
): TimeRange[] {
  let free = [...openingRanges]
  for (const booked of bookedRanges) {
    free = free.flatMap((range) => range.subtract(booked))
  }
  return free
}
```

Fragmento 5.1. Resta acumulativa de reservas sobre los tramos de apertura (`availability-calculator.ts`).

Sobre el conjunto de rangos libres resultante, una segunda función, `canFitService`, determina si un servicio de duración dada **cabe** en el hueco que el cliente ha seleccionado, reutilizando el método `contains` del objeto de valor:

```
export function canFitService(
  startMinute: number,
  durationMinutes: number,
  freeRanges: TimeRange[]
): boolean {
  const needed = new TimeRange(startMinute, startMinute + durationMinutes)
  return freeRanges.some((free) => free.contains(needed))
}
```

Fragmento 5.2. Verificación de ajuste de un servicio en los huecos libres (`availability-calculator.ts`).

La representación del tiempo como **minutos desde la medianoche** (un entero en `[0, 1440]`, donde 1 440 designa el final de la jornada y, en consecuencia, solo puede actuar como extremo de cierre de un intervalo —el inicio de una reserva queda en `[0, 1440)` —) es una decisión de diseño deliberada: convierte el solapamiento, la resta y la contención de intervalos en aritmética

entera trivial, libre de las ambigüedades de las marcas de tiempo absolutas. La conversión desde y hacia el formato `HH:MM` se encapsula en el propio `TimeRange` (`parseHHMM`).

5.3. Integridad de las reservas ante la concurrencia

El cálculo de disponibilidad de la sección anterior es **optimista**: comprueba el estado del sistema en un instante dado. Sin embargo, entre el momento en que un cliente consulta la disponibilidad y el momento en que confirma la reserva, **otro cliente puede haber reservado el mismo hueco**. Una validación realizada únicamente en la capa de aplicación sería vulnerable a esta condición de carrera (*race condition*): dos peticiones simultáneas podrían superar ambas la comprobación `canFitService` y persistir reservas solapadas.

La solución adoptada lleva la garantía de integridad a la **única capa verdaderamente autoritativa frente a la concurrencia: la base de datos**. Se define una **restricción de exclusión** (`EXCLUDE`) de PostgreSQL, habilitada por la extensión `btree_gist`, que el motor evalúa de forma atómica en cada inserción (`supabase/migrations/20260422_prevent_booking_overlap.sql`):

```
create extension if not exists btree_gist;

alter table public.bookings
  add constraint bookings_no_overlap
  exclude using gist (
    tenant_id with =,
    date with =,
    int4range(start_minutes, end_minutes, '[]') with &&
  )
  where (status <> 'CANCELLED');
```

Fragmento 5.3. Restricción de exclusión que impide solapamientos a nivel de base de datos.

La semántica de la restricción se lee así: el sistema rechazará toda nueva reserva que comparta el mismo `tenant_id` (`with =`) y la misma `date` (`with =`) y cuyo intervalo `[start_minutes, end_minutes)` se **solape** (`with &&`) con el de una reserva existente. Tres decisiones merecen comentario:

- El intervalo se modela como `int4range` **semiabierto** `[]'`: una reserva que termina en el minuto 60 y otra que empieza en el minuto 60 **no** se consideran solapadas, lo que es el comportamiento correcto para citas consecutivas.
- La cláusula `where (status <> 'CANCELLED')` es un **índice parcial**: las reservas canceladas se excluyen de la restricción, de modo que un hueco liberado puede volver a reservarse.
- La combinación de la igualdad (`=`, propia de un índice B-tree) con el solapamiento de rangos (`&&`, propio de un índice GiST) es precisamente lo que exige la extensión `btree_gist`, y constituye una capacidad que un almacén documental no relacional no podría expresar (véase el Capítulo 2).

Cuando esta restricción se viola, PostgreSQL aborta la inserción con el **código de error** `23P01` (*exclusion_violation*). El adaptador de persistencia lo intercepta y lo **traduce a un error de dominio** tipado, `SlotTakenError`, preservando así la regla de que las capas superiores nunca dependen de detalles del motor:

```
if (error) {
  // 23P01 = exclusion_violation (Postgres): the bookings_no_overlap
  // constraint rejected this insert because another booking already
  // occupies an overlapping time range for the same tenant+date.
  if (error.code === '23P01') {
    throw new SlotTakenError()
  }
  throw new Error(`Failed to save booking: ${error.message}`)
}
```


El resultado es una **defensa en dos niveles**: la comprobación optimista en la capa de aplicación ofrece una respuesta inmediata y mensajes de error precisos en el caso común, mientras que la restricción de la base de datos actúa como **garantía pesimista e inviolable** en el caso límite de la concurrencia. La `server action` pública captura `SlotTakenError` y comunica al usuario, con elegancia, que el hueco acaba de ocuparse.

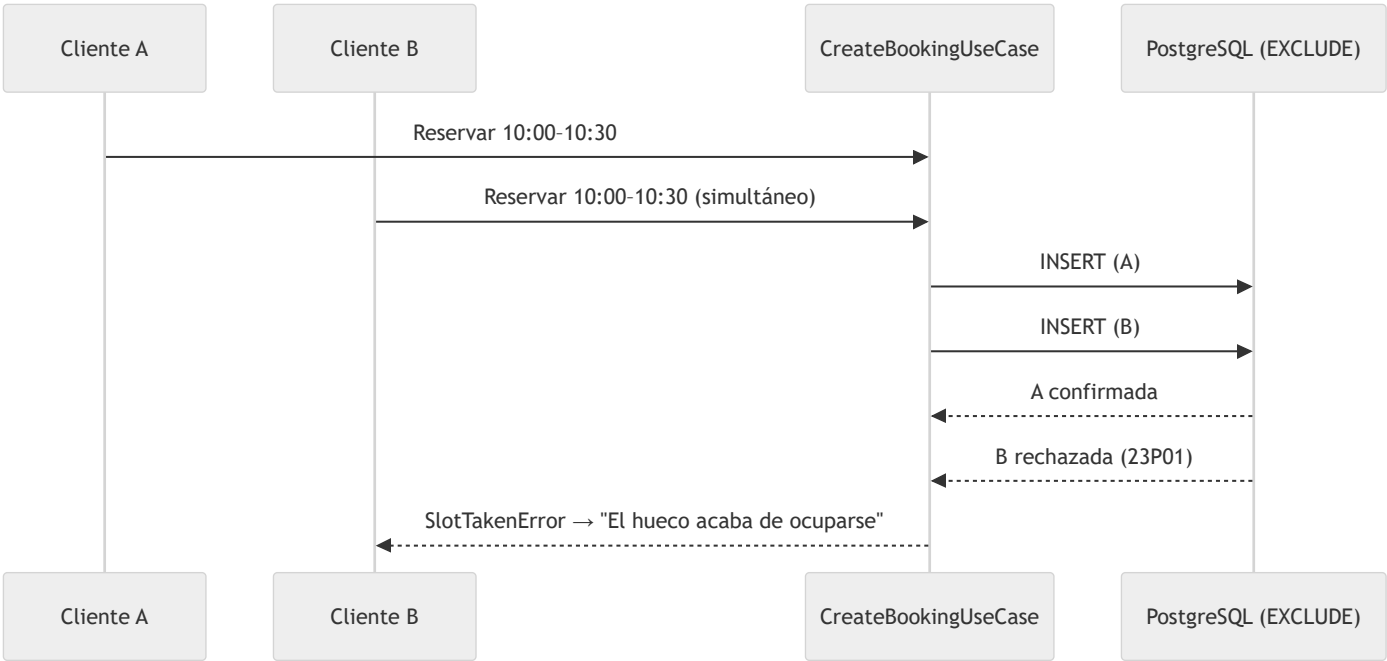


Figura 5.1. La restricción de exclusión resuelve la condición de carrera de forma atómica: solo una de dos reservas concurrentes idénticas prospera.

5.4. Gestión sensible a la zona horaria y política de antelación

Un sistema de citas debe razonar sobre el tiempo **en la zona horaria del negocio**, no en la del servidor (que en un entorno *serverless* opera en UTC). El servicio de dominio `tenant-clock` (`domain/services/tenant-clock.ts`) concentra estas operaciones apoyándose en la API estándar `Intl` a través de `toLocaleDateString` / `toLocaleTimeString` con el parámetro `timeZone` :

```
/** Returns minutes from midnight in the tenant's timezone (0-1439) */
export function getTenantLocalMinutes(timezone: string, now?: Date): number {
  const d = now ?? new Date()
  const parts = d
    .toLocaleTimeString('en-GB', {
      timeZone: timezone,
      hour: '2-digit',
      minute: '2-digit',
      hour12: false,
    })
    .split(':')
  return Number(parts[0]) * 60 + Number(parts[1])
}
```

El uso del locale `'en-CA'` para la fecha (`getTenantLocalDate`) es un detalle deliberado: produce el formato `YYYY-MM-DD` , directamente comparable con las fechas que maneja el sistema. Estas funciones sostienen la **política de antelación** del negocio,

aplicada en `CreateBookingUseCase` : una **antelación máxima** (`maxAdvanceDays` , no se puede reservar con demasiada anticipación), la prohibición de **fechas pasadas** y, para las reservas del día en curso, una **antelación mínima** (`minAdvanceMinutes`) calculada contra la hora local real del negocio. La firma del caso de uso admite además un parámetro opcional `now?: Date` , lo que permite **inyectar el instante actual** en las pruebas y verificar de forma determinista todas las fronteras temporales.

Se documenta, en aras del rigor, una **limitación conocida**: la derivación del día de la semana a partir de la fecha seleccionada emplea `getUTCDay()` , decisión que puede producir desviaciones para *tenants* en husos alejados de UTC en condiciones límite. Su corrección se recoge entre las líneas futuras del Capítulo 7.

5.5. Integración de pagos B2B2C con Stripe Connect

La naturaleza *marketplace* del producto se materializa en el flujo de pago. Cuando un cliente abona un servicio, el dinero debe llegar a la cuenta del **negocio**, no a la de la plataforma, reteniendo esta una **comisión**. Esto se implementa con **Stripe Connect** y dos mecanismos concretos de su API: la **cuenta de destino** (`stripeAccount`) y la **comisión de aplicación** (`application_fee_amount`).

El **alta de la cuenta conectada** del negocio se resuelve con **Stripe Connect Express**: la plataforma crea la cuenta por API (`accounts.create` con `type: 'express'`) y redirige al comercio a un **onboarding alojado por Stripe** (`accountLinks.create` , un enlace de un solo uso) donde este aporta y verifica su identidad y sus datos bancarios; al regresar, el sistema consulta `charges_enabled` para habilitar el cobro. El caso de uso `StripeOnboardingUseCase` orquesta ambas fases —iniciar el *onboarding* y sincronizar el estado—. Este modelo minimiza la fricción del comercio y evita que la plataforma gestione el redireccionamiento OAuth y sus credenciales.

```
const feeAmount = Math.round(
  (request.price.amountCents * request.commissionRateBps) / 10000
)

const session = await getStripe().checkout.sessions.create(
  {
    mode: 'payment',
    /* ... line_items ... */
    payment_intent_data: {
      application_fee_amount: feeAmount,
    },
    metadata: {
      bookingId: request.bookingId,
    },
    success_url: request.successUrl,
    cancel_url: request.cancelUrl,
  },
  { stripeAccount: request.stripeAccountId }
)
```

Fragmento 5.6. Creación de la sesión de pago sobre la cuenta conectada del negocio, con comisión de plataforma (`payment-service.ts`).

La comisión se calcula en **puntos básicos** (`commissionRateBps`): el servicio de dominio `plan-limits` define 500 bps (5 %) para el plan `FREE` y 100 bps (1 %) para el plan `PRO` . El cálculo `amountCents * bps / 10000` con `Math.round` evita el uso de aritmética de coma flotante sobre importes monetarios. El segundo argumento, `{ stripeAccount: request.stripeAccountId }` , instruye a Stripe para que el cargo se efectúe **en nombre de la cuenta conectada** del negocio.

La **confirmación** de la reserva no se produce al crear la sesión, sino de forma **asíncrona y verificada** cuando Stripe notifica el evento `checkout.session.completed` a través de un *webhook*. El manejador **valida criptográficamente la firma** de la notificación antes de actuar, y solo confirma la reserva si esta sigue en estado `PENDING` —una guarda de **idempotencia** que evita confirmaciones o correos duplicados ante reintentos de Stripe—²⁶ / 40

```

event = stripe.webhooks.constructEvent(
  body,
  signature,
  process.env.STRIPE_CONNECT_WEBHOOK_SECRET!
)
/* ... */
const booking = await bookingRepo.findById(bookingId)
if (booking && booking.status === BookingStatus.PENDING) {
  await bookingRepo.updateStatus(bookingId, BookingStatus.CONFIRMED)
  await sendConfirmationEmails(supabase, bookingId)
}

```

Fragmento 5.7. Confirmación idempotente de la reserva tras verificar la firma del webhook (`webhooks/stripe-connect/route.ts`).

Conviene matizar que el modelo de planes (`FREE / PRO`) está **diseñado en el dominio pero no es funcional todavía**: como se señaló en el Capítulo 4, no existe columna `plan` en el esquema, por lo que todo negocio resuelve a `FREE` y la comisión efectiva es siempre del 5 %. La activación del plan `PRO` figura como trabajo futuro.

5.6. Recordatorios idempotentes: el patrón *claim-and-release*

El envío de recordatorios lo dispara una **tarea programada** (`api/cron/send-reminders`) protegida por un secreto en la cabecera `Authorization` . El reto es la **idempotencia**: si el proceso se ejecuta de forma solapada, o falla a mitad de un lote, ninguna reserva debe recibir dos recordatorios. La solución es un patrón **reclamar-y-liberar** (*claim-and-release*) sustentado en una operación atómica de **comparar-e-intercambiar** (*compare-and-swap*) sobre la propia base de datos:

```

async claimReminder(id: string, sentAt: Date): Promise<boolean> {
  const { data, error } = await this.supabase
    .from('bookings')
    .update({ reminder_sent_at: sentAt.toISOString() })
    .eq('id', id)
    .is('reminder_sent_at', null) // solo si nadie lo ha reclamado aún
    .select('id')

  if (error) throw new Error(`Failed to claim reminder: ${error.message}`)
  return (data ?? []).length > 0
}

```

Fragmento 5.8. Reclamación atómica del envío mediante un update condicionado (`booking-repository.ts`).

La cláusula `.is('reminder_sent_at', null)` es la clave: el `UPDATE` solo afecta a la fila si el recordatorio **aún no ha sido reclamado**, y devuelve si la operación tuvo éxito. El planificador procede entonces así: para cada reserva candidata, intenta **reclamarla**; si la reclamación falla, otra ejecución ya la atendió y la **omite**; si la reclamación tiene éxito pero el envío del correo **falla, libera** la reclamación (`releaseReminder`) para que un ciclo posterior la reintente. De este modo se satisface el requisito no funcional RNF-8 sin necesidad de un sistema de colas externo.

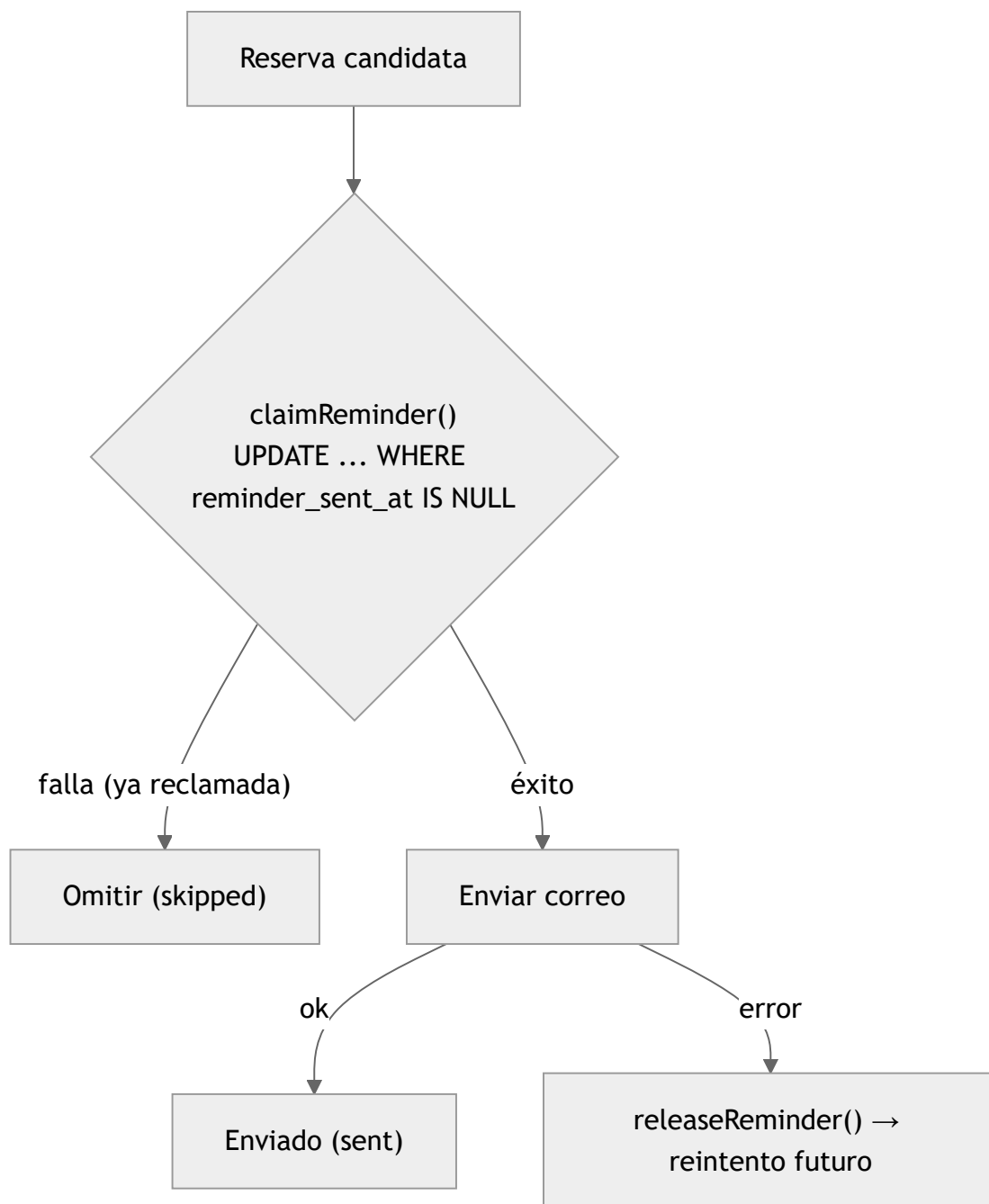


Figura 5.2. Patrón claim-and-release: la reclamación atómica garantiza, como máximo, un recordatorio por reserva.

5.7. Multi-tenancy y seguridad a nivel de fila (RLS)

El aislamiento entre negocios se apoya en la **seguridad a nivel de fila** (*Row Level Security*) de PostgreSQL, habilitada en las cinco tablas. El patrón general consiste en conceder al **propietario** acceso completo a las filas de su negocio, comprobando la pertenencia contra `auth.uid()`:

```
create policy "Owner full access" on public.services
for all using (
  tenant_id in (select id from public.tenants where owner_id = auth.uid())
);
```

Fragmento 5.9. Política RLS de acceso del propietario a sus propios servicios (`20260220221052_initial_schema.sql`).

Este patrón se aplica de forma robusta a `tenants`, `services` y `schedules`. Ahora bien, en coherencia con el principio de **documentación honesta de las limitaciones** que vertebrará este trabajo, debe señalarse una **brecha real de seguridad** en

el esquema actual: las tablas `bookings` y `customers` definen políticas **deliberadamente permisivas**, con lectura e inserción públicas sin restricción:

```
create policy "Public insert" on public.customers for insert with check (true);
create policy "Public read"    on public.customers for select using (true);
-- ídem para bookings
```

Fragmento 5.10. Políticas permisivas que constituyen la principal deuda de seguridad del sistema
(`20260220221052_initial_schema.sql`).

Esta permisividad fue una concesión pragmática para habilitar el flujo de reserva **anónimo** (un cliente final sin sesión debe poder crear una reserva y su registro de cliente). Su efecto colateral es que **el aislamiento estricto de datos entre *tenants* no está garantizado a nivel de fila** para estas dos tablas: un acceso directo al cliente de datos podría, en teoría, leer reservas o datos de clientes de otros negocios. En la práctica, el código de aplicación siempre filtra por `tenant_id`, pero la garantía **no está, como debería, en la capa declarativa de la base de datos**. La subsanación de esta brecha —reescribir las políticas para acotar el acceso anónimo a la operación estrictamente necesaria— es la primera de las líneas prioritarias del Capítulo 7 y alimentaría un capítulo de modelado de amenazas.

5.8. Internacionalización

El sistema ofrece soporte bilingüe (es-ES / en-US) en dos planos. En el **panel de administración**, la traducción se resuelve en el servidor (`infrastructure/i18n/`), evitando enviar funciones como propiedades a los componentes cliente —una restricción real de los React Server Components que motivó una corrección específica durante el desarrollo—. En las **comunicaciones por correo** (`infrastructure/resend/`), la plantilla, su traducción y el formateo de fechas se seleccionan según el locale del negocio, manteniendo el envío detrás del puerto `NotificationService` de la capa de aplicación.

Un tercer plano —los **correos de autenticación** (confirmación de cuenta, restablecimiento de contraseña)— presentaba una restricción propia: las plantillas de Supabase Auth son de un solo idioma. Se resolvió con un ***Send Email Hook***: Supabase delega el envío en un endpoint propio (`api/auth/send-email`) que **verifica la firma HMAC** (Standard Webhooks) de la petición, resuelve el idioma del usuario —guardado en sus metadatos al registrarse— y renderiza el correo con la misma infraestructura `i18n`, enviándolo por Resend desde el dominio verificado. De este modo, **todo el correo del sistema** —transaccional y de cuenta— es bilingüe y coherente en remitente.

5.9. Síntesis

Los seis aspectos detallados comparten un mismo principio rector: **situar cada garantía en la capa adecuada**. La lógica de disponibilidad reside en el dominio puro y comprobable; la integridad ante la concurrencia se delega en la capa autoritativa de la base de datos mediante una restricción de exclusión; la confirmación del pago se ancla en un *webhook* firmado e idempotente; y la idempotencia de los recordatorios se logra con una operación atómica de comparar-e-intercambiar. El capítulo ha expuesto asimismo, sin omitirlas, las dos limitaciones más relevantes —la permisividad de las políticas RLS sobre `bookings / customers` y la inoperatividad del modelo de planes—, que se retoman como trabajo futuro. La estrategia de pruebas que verifica todos estos comportamientos se describe en el Capítulo 6.

Capítulo 6. Estrategia de Pruebas y Control de Calidad

6.1. Filosofía: TDD y la pirámide de pruebas

La calidad del software es, según se argumentó en el Capítulo 1, el eje vertebrador del Trabajo de Fin de Máster. Su materialización más tangible es la **batería de pruebas automatizadas**, concebida bajo dos principios complementarios:

- **Desarrollo guiado por pruebas (*Test-Driven Development*)**: en las capas de dominio y aplicación, la prueba precede a la implementación, de modo que cada regla de negocio nace acompañada de su especificación ejecutable.
- **La pirámide de pruebas**: la mayor densidad de pruebas se concentra en la base —pruebas unitarias rápidas y deterministas sobre el dominio—, disminuyendo a medida que se asciende hacia la infraestructura, más costosa de verificar. Esta forma no es casual: es la consecuencia directa de una Arquitectura Limpia que aísla la lógica de negocio de sus dependencias.

6.2. Marco y configuración

El marco de pruebas es **Vitest 4** (`vitest run` para la ejecución única; `vitest` en modo vigilancia). La configuración integra `vite-tsconfig-paths` —que resuelve el alias de módulos `@/*` también en las pruebas— y `@vitejs/plugin-react`. La elección frente a Jest se justificó en el Capítulo 2 por su compatibilidad nativa con ESM y TypeScript. El proyecto define además un script de cobertura (`test:coverage`) con un **umbral que opera como puerta de calidad automatizada** (§6.7).

6.3. Distribución de la batería de pruebas

La suite comprende **240 casos de prueba distribuidos en 32 ficheros**. Su reparto por capas evidencia la pirámide descrita:

Capa	Ficheros de prueba	Casos	Peso
Dominio	11	123	51 %
Aplicación (casos de uso)	7	57	24 %
Infraestructura	14	60	25 %
Total	32	240	100 %

Tabla 6.1. Distribución de la batería de pruebas por capa arquitectónica.

El dato relevante no es solo el volumen, sino su **forma**: el 75 % de las pruebas se concentra en las capas de dominio y aplicación — las que albergan las reglas de negocio—, y el dominio por sí solo es la capa más ejercitada (51 %), lo que constituye una evidencia objetiva de que la arquitectura ha cumplido su propósito de hacer la lógica crítica verificable de forma aislada y barata.

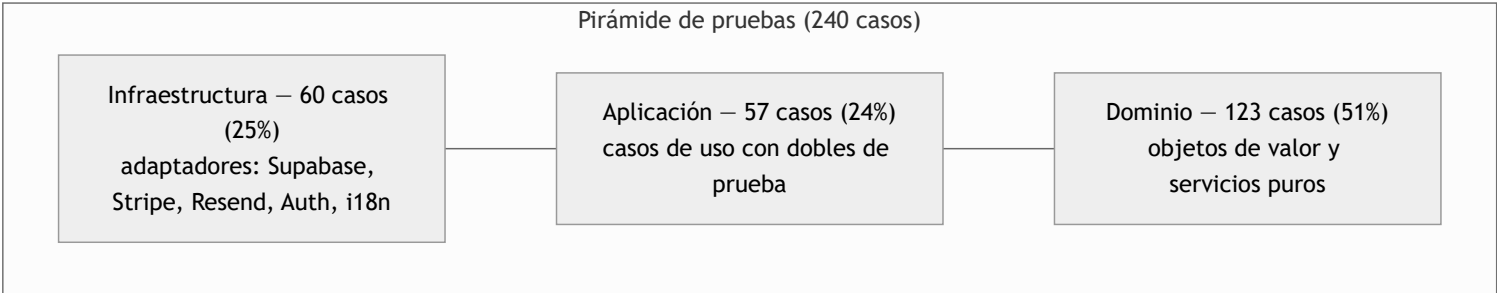


Figura 6.1. La batería de pruebas adopta la forma de pirámide: base ancha en el dominio puro.

6.4. Pruebas de la capa de dominio

El dominio, al carecer de dependencias externas, se presta a una verificación **exhaustiva y determinista**. Los objetos de valor concentran buena parte del esfuerzo: `TimeRange` reúne por sí solo 27 casos que ejercitan sus invariantes (rechazo de rangos inválidos) y su comportamiento (`overlaps` , `contains` , `subtract`). Las pruebas del `availability-calculator` (15 casos) verifican el algoritmo de resta de reservas frente a casos representativos —reserva en el centro de un tramo que lo parte en dos, reservas que dejan el horario completo, encadenamientos—, que son precisamente los escenarios en los que un cálculo ingenuo fallaría.

6.5. Pruebas de la capa de aplicación

Los casos de uso se prueban **de forma aislada**, sustituyendo sus colaboradores (repositorios y servicios) por **dobles de prueba** (*test doubles*) que implementan los mismos puertos. Esto es posible, sin artificios, gracias a la **inyección de dependencias por constructor** descrita en el Capítulo 4: `CreateBookingUseCase` recibe cinco repositorios como interfaces y, en las pruebas, se le inyectan implementaciones en memoria.

Dos decisiones de diseño habilitan un determinismo total:

- La inyección del instante actual (`now?: Date`) permite fijar «la hora del sistema» en cada prueba, verificando las fronteras de la **política de antelación** (reserva demasiado pronto, demasiado tarde, en el pasado) sin depender del reloj real.
- La separación de responsabilidades hace que cada una de las **excepciones de dominio** (`TenantNotFoundError` , `BusinessClosedError` , `ServiceDoesNotFitError` , etc.) sea un caso de prueba dirigido y reproducible.

Los 13 casos de `create-booking.test.ts` cubren así tanto el flujo satisfactorio como el conjunto de ramas de error del caso de uso más complejo del sistema.

6.6. Pruebas de la capa de infraestructura

En la infraestructura, las pruebas verifican la **correcta traducción entre el mundo externo y el dominio**, no las dependencias de terceros en sí. Son representativos dos casos:

- En `payment-service.test.ts` , se comprueba que el **cálculo de la comisión** en puntos básicos (`amountCents * bps / 10000`) es correcto y que la sesión de pago se construye con los parámetros esperados de Stripe Connect.
- En `booking-repository.test.ts` , se verifica que el código de error `23P01` de PostgreSQL se traduce efectivamente a `SlotTakenError` , garantizando que la frontera de integridad descrita en el Capítulo 5 se comporta como se espera.

6.7. Análisis estático, integración continua y cobertura

Más allá de las pruebas dinámicas, el control de calidad se apoya en **dos líneas de análisis estático**: el **compilador de TypeScript en modo estricto** (`tsc`), que erradica clases enteras de errores en tiempo de compilación, y **ESLint** con la configuración de Next.js (`eslint-config-next`), ambos sin errores.

Estas comprobaciones no son solo locales: un flujo de **integración continua** (GitHub Actions, `.github/workflows/ci.yml`) las institucionaliza como **puerta de calidad**, ejecutando en cada *push* y *pull request* la secuencia `lint → tsc --noEmit → vitest --coverage → build` . La **cobertura actúa como puerta**: el umbral configurado (85 % de sentencias, funciones y líneas; 75 % de ramas) hace **fallar la construcción** si no se alcanza, medido sobre las capas de dominio, aplicación e infraestructura.

La capa de **presentación** (rutas, *Server Actions* y componentes), cuya cobertura por líneas sería engañosa al medir dobles del *framework* más que lógica propia, se valida mediante **pruebas end-to-end automatizadas con Playwright** (`e2e/` , `script test:e2e`), que recorren el flujo real de reserva contra el despliegue. Con ello, la calidad **practicada** se convierte en calidad **medida y forzada por la herramienta**.

6.8. Síntesis

La estrategia de pruebas no es un añadido posterior, sino el reflejo directo de la arquitectura: una pirámide de 240 casos cuya base ancha (75 % en dominio y aplicación) solo es posible porque la lógica de negocio se diseñó desacoplada y comprobable. El análisis estático estricto, la cobertura acotada por **umbral**, las pruebas **E2E automatizadas** y su ejecución en **integración continua** completan un marco de calidad no solo *practicado*, sino *medido y forzado por la herramienta*.

Capítulo 7. Conclusiones y Líneas Futuras

7.1. Grado de cumplimiento de los objetivos

El presente trabajo se propuso, en su Capítulo 1, un objetivo general doble —construir una plataforma SaaS de reservas de calidad de producción y, simultáneamente, validar un proceso de desarrollo asistido por IA centrado en la calidad— desplegado en una serie de objetivos específicos y metodológicos. La siguiente tabla valora su grado de cumplimiento frente a la evidencia del repositorio.

Objetivo	Estado	Evidencia
OE-1. Arquitectura Limpia con capas estrictas	Cumplido (con desviación acotada)	<code>src/domain</code> , <code>application</code> , <code>infrastructure</code> , <code>app</code> ; dominio sin dependencias de <i>framework</i> . Desviación: rama de administración (§4.5)
OE-2. Modelado del dominio (entidades, VO, servicios)	Cumplido	5 objetos de valor con invariantes; servicios <code>availability-calculator</code> , <code>tenant-clock</code> , <code>plan-limits</code>
OE-3. Integridad ante concurrencia en la BD	Cumplido	Restricción <code>EXCLUDE / btree_gist → SlotTakenError</code> (§5.3)
OE-4. Multi-tenancy con RLS	Parcialmente cumplido	RLS en 5 tablas; políticas permisivas en <code>bookings / customers</code> (§5.7)
OE-5. Pagos B2B2C con Stripe Connect	Cumplido (monetización inactiva)	Cuenta de destino + comisión; confirmación por <i>webhook</i> firmado. Plan <code>PRO</code> no persistido (§5.5)
OE-6. Internacionalización (es-ES/en-US)	Cumplido	<code>infrastructure/i18n</code> , correos traducidos
OE-7. Despliegue <i>serverless</i>	Cumplido	Vercel; <code>https://reservas-chanantes.vercel.app/</code>
OM-1. TDD en dominio y aplicación	Cumplido	240 pruebas; 75 % en dominio/aplicación, cobertura con umbral, CI y E2E (§6.7)
OM-2. Trazabilidad del proceso asistido por IA	Cumplido	<code>docs/plans/</code> , <code>docs/reviews/</code> , <code>mvp-deviations.md</code>
OM-3. Documentación honesta de limitaciones	Cumplido	Capítulos 4–7

Tabla 7.1. Grado de cumplimiento de los objetivos planteados.

El balance es **mayoritariamente satisfactorio**: de diez objetivos, siete se cumplen plenamente, dos lo hacen de forma parcial pero honestamente acotada (RLS y monetización) y uno presenta una desviación arquitectónica documentada (acceso directo a repositorios en administración). Ninguna de estas brechas compromete el funcionamiento del producto en su alcance de MVP, y todas se reconocen explícitamente —lo que, lejos de restar valor, refuerza el objetivo metodológico OM-3.

7.2. Valoración metodológica

El verdadero objeto de estudio del trabajo —la **viabilidad del desarrollo de software guiado por IA priorizando la máxima calidad**— admite una valoración positiva sustentada en hechos verificables, no en impresiones:

- La **arquitectura desacoplada** no es declarativa, sino comprobable: el dominio no importa ninguna dependencia de *framework*, y esa propiedad habilitó una pirámide de pruebas de base ancha.
- La **calidad se tradujo en artefactos**: 240 pruebas, 10 migraciones versionadas, y una trazabilidad del proceso (planes, revisiones, registro de desviaciones) que documenta no solo *qué* se construyó, sino *cómo* se decidió.
- La elección de soluciones **técnicamente exigentes y correctas** —la restricción de exclusión para la concurrencia, el patrón *claim-and-release* para la idempotencia, el modelado del tiempo como aritmética entera— demuestra que el proceso asistido por IA fue capaz de abordar la complejidad esencial del dominio, no solo la accidental.

La conclusión metodológica es que el desarrollo asistido por IA, **conducido bajo una disciplina de ingeniería explícita**, es viable para producir software de calidad de producción; y que su principal riesgo —la generación de código plausible pero

superficial— se mitiga precisamente con los mecanismos que este trabajo adoptó: TDD, fronteras arquitectónicas estrictas y revisión documentada.

7.3. Limitaciones del trabajo

Se consolidan aquí, de forma transparente, las limitaciones señaladas a lo largo de la memoria:

1. **Aislamiento multi-tenant incompleto:** las políticas RLS de `bookings` y `customers` son permisivas (§5.7).
2. **Monetización no funcional:** ausencia de columna `plan` ; todo *tenant* resuelve a `FREE` (§5.5).
3. **Arquitectura Limpia parcial en presentación:** los *Server Actions* de administración acceden directamente a los repositorios (§4.5).
4. **Defecto conocido de zona horaria:** derivación del día de la semana con `getUTCDay()` (§5.4).
5. **Sin pruebas de integración contra base de datos real:** los adaptadores de persistencia se prueban con dobles (*mocks*); la restricción `EXCLUDE` y las políticas RLS se validan a ese nivel solo indirectamente, no contra un PostgreSQL efímero (§6.7, Anexo F). (*El resto del marco de calidad —CI, E2E automatizado y umbral de cobertura— ya está en marcha.*)
6. **Observabilidad mínima:** el registro se limita a `console.error` , sin trazas estructuradas ni métricas.

7.4. Líneas futuras

Las líneas de evolución se priorizan por su **retorno** —tanto de producto como académico—.

7.4.1. Prioridad alta (refuerzo de la calidad declarada)

1. **Endurecimiento de la seguridad y la multi-tenancy.** Reescribir las políticas RLS de `bookings` / `customers` para acotar el acceso anónimo a lo estrictamente necesario; añadir **claves de idempotencia** en las operaciones de Stripe; escapar el HTML en los correos; e introducir limitación de tasa (*rate limiting*). Este conjunto —cuya evaluación por categoría OWASP figura en el [Anexo E](#)— habilitaría un **capítulo de modelado de amenazas**.
2. **Pruebas de integración contra base de datos real.** La integración continua (GitHub Actions: `lint` → `tsc` → `vitest` - `-coverage` con umbral), las **pruebas E2E de Playwright** y el gate de cobertura ya están operativos (§6.7); el siguiente refuerzo natural es ejercitar la restricción `EXCLUDE` y las políticas RLS **de extremo a extremo contra un PostgreSQL efímero**, hoy verificadas solo con dobles de prueba.

7.4.2. Prioridad media (completar el producto)

3. **Activar la monetización:** añadir la columna `plan` , su flujo de suscripción y la aplicación efectiva de límites y comisiones por plan, dando vida al servicio `plan-limits` ya diseñado.
4. **Sanear la coherencia arquitectónica:** encauzar los *Server Actions* de administración a través de casos de uso, eliminando el acceso directo a repositorios.
5. **Corregir el defecto de zona horaria** en la derivación del día de la semana.

7.4.3. Prioridad baja (madurez operativa)

6. **Observabilidad:** incorporar trazas estructuradas y métricas.
7. **Capítulo de métricas de metodología:** cuantificar el desarrollo asistido por IA (cobertura por fase, defectos detectados en revisión, *commits* por funcionalidad) a partir de los artefactos de `docs/` .

7.5. Conclusión final

El proyecto **Reservas Chanantes** alcanza su doble objetivo: entrega una plataforma SaaS multi-tenant de reservas funcional y desplegada, y lo hace sobre una base de ingeniería —Arquitectura Limpia, TDD, integridad garantizada en la base de datos— cuya solidez es verificable en el código y en una batería de 240 pruebas. Igual de relevante para un Trabajo de Fin de Máster es que sus limitaciones se han identificado y documentado con honestidad, trazando un camino de evolución claro. El trabajo sostiene, en definitiva, su tesis central: el desarrollo de software guiado por Inteligencia Artificial, cuando se somete a una disciplina de calidad explícita, es una vía viable para construir sistemas no triviales y bien fundamentados.

Bibliografía

Las referencias se presentan en formato **IEEE**. Las citas numéricas entre corchetes que aparecen en el cuerpo de la memoria ([n]) remiten a la sección de **referencias académicas**. La **documentación técnica consultada** recoge las fuentes primarias de las tecnologías y normas empleadas, citadas a lo largo de los capítulos 2 a 7.

Referencias académicas

[1] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston, MA, USA: Prentice Hall, 2017.

[2] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.

[3] A. Cockburn, "Hexagonal Architecture (Ports and Adapters)," 2005. [En línea]. Disponible: <https://alistair.cockburn.us/hexagonal-architecture/>

[4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[5] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2002.

[6] R. Krebs, C. Momm y S. Kounev, "Architectural Concerns in Multi-Tenant SaaS Applications," en *Proc. 2nd Int. Conf. on Cloud Computing and Services Science (CLOSER 2012)*, SciTePress, 2012, pp. 426–431.

[7] React Team, "Server Components," *React Documentation*. [En línea]. Disponible: <https://react.dev/reference/rsc/server-components>

Documentación técnica y normas consultadas

[8] Vercel Inc., "Next.js Documentation — App Router," 2026. [En línea]. Disponible: <https://nextjs.org/docs>

[9] The PostgreSQL Global Development Group, "Constraints — Exclusion Constraints," *PostgreSQL Documentation*, 2026. [En línea]. Disponible: <https://www.postgresql.org/docs/current/ddl-constraints.html>

[10] The PostgreSQL Global Development Group, "btree_gist," *PostgreSQL Documentation*, 2026. [En línea]. Disponible: <https://www.postgresql.org/docs/current/btree-gist.html>

[11] Supabase Inc., "Row Level Security," *Supabase Documentation*, 2026. [En línea]. Disponible: <https://supabase.com/docs/guides/database/postgres/row-level-security>

[12] Stripe Inc., "Stripe Connect — Platforms and Marketplaces," *Stripe Documentation*, 2026. [En línea]. Disponible: <https://docs.stripe.com/connect>

[13] Vercel Inc., "Cron Jobs," *Vercel Documentation*, 2026. [En línea]. Disponible: <https://vercel.com/docs/cron-jobs>

[14] Resend Inc., "Resend Documentation," 2026. [En línea]. Disponible: <https://resend.com/docs>

[15] Vitest Team, "Vitest — Next Generation Testing Framework," 2026. [En línea]. Disponible: <https://vitest.dev/>

[16] J. Klensin, "Simple Mail Transfer Protocol," RFC 5321, IETF, oct. 2008. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc5321>

[17] OWASP Foundation, "OWASP Top 10 — 2021," 2021. [En línea]. Disponible: <https://owasp.org/Top10/>

Anexos

Los anexos recogen el detalle estructural del sistema, complementando los capítulos 4 y 5. Toda la información se ha extraído directamente de las migraciones versionadas (`supabase/migrations/`) y del código fuente.

Anexo A. Diccionario de datos

Esquema relacional completo tras aplicar las diez migraciones. Tipos y restricciones en notación PostgreSQL.

Tabla tenants

Columna	Tipo	Restricciones y notas
id	uuid	PK, default gen_random_uuid()
owner_id	uuid	FK → auth.users(id) ON DELETE CASCADE , NOT NULL
name	text	NOT NULL
slug	text	UNIQUE , NOT NULL
currency	text	NOT NULL , default 'EUR' , CHECK IN ('EUR' , 'USD' , 'GBP')
default_locale	text	NOT NULL , default 'es-ES' , CHECK IN ('es-ES' , 'en-US')
created_at	timestamptz	NOT NULL , default now()
timezone	text	NOT NULL , default 'Europe/Madrid'
min_advance_minutes	integer	NOT NULL , default 120 , CHECK >= 0
max_advance_days	integer	NOT NULL , default 30 , CHECK >= 1
stripe_account_id	text	Índice único parcial (WHERE NOT NULL)
stripe_account_enabled	boolean	NOT NULL , default false
description	text	Perfil de negocio
category	text	NOT NULL , default 'LocalBusiness'
city	text	Perfil de negocio
address	text	Perfil de negocio
phone	text	Teléfono de contacto del negocio
seo_title	text	Personalización SEO
seo_description	text	Personalización SEO

Tabla services

Columna	Tipo	Restricciones y notas
id	uuid	PK, default gen_random_uuid()
tenant_id	uuid	FK → tenants(id) ON DELETE CASCADE , NOT NULL
name	text	NOT NULL
duration_minutes	integer	NOT NULL , CHECK > 0
price_cents	integer	NOT NULL , default 0 , CHECK >= 0
active	boolean	NOT NULL , default true

Tabla schedules

Columna	Tipo	Restricciones y notas
id	uuid	PK, default gen_random_uuid()
tenant_id	uuid	FK → tenants(id) ON DELETE CASCADE , NOT NULL
day_of_week	integer	NOT NULL , CHECK BETWEEN 0 AND 6 (0 = domingo)
time_ranges	jsonb	NOT NULL , default '[]' ; lista de {start, end} en minutos
—	—	UNIQUE (tenant_id, day_of_week)

Tabla customers

Columna	Tipo	Restricciones y notas
id	uuid	PK, default gen_random_uuid()
name	text	NOT NULL
email	text	UNIQUE , NOT NULL (identidad transversal del cliente)
phone	text	NOT NULL , default '' (originalmente nullable)
auth_user_id	uuid	FK → auth.users(id) ON DELETE SET NULL ; índice único parcial
preferred_locale	text	CHECK IN ('es-ES', 'en-US')

El cliente **no** tiene `tenant_id` : se identifica por `email` y puede tener reservas en varios negocios. La vinculación con una cuenta autenticada se realiza mediante `auth_user_id` (portal del cliente).

Tabla bookings

Columna	Tipo	Restricciones y notas
id	uuid	PK, default gen_random_uuid()
tenant_id	uuid	FK → tenants(id) ON DELETE CASCADE , NOT NULL
service_id	uuid	FK → services(id) ON DELETE CASCADE , NOT NULL
customer_id	uuid	FK → customers(id) ON DELETE CASCADE , NOT NULL
date	date	NOT NULL
start_minutes	integer	NOT NULL , CHECK (>= 0 AND < 1440)
end_minutes	integer	NOT NULL , CHECK (> 0 AND <= 1440)
status	text	NOT NULL , default 'PENDING' , CHECK IN ('PENDING', 'CONFIRMED', 'CANCELLED')
created_at	timestamptz	NOT NULL , default now()
stripe_checkout_session_id	text	Índice parcial (WHERE NOT NULL)
reminder_sent_at	timestamptz	Marca de recordatorio enviado; índice parcial
—	—	CHECK (start_minutes < end_minutes)
—	—	EXCLUDE bookings_no_overlap (véase Anexo A.1)

A.1. Restricción de exclusión bookings_no_overlap

```
exclude using gist (  
  tenant_id with =,  
  date with =,  
  int4range(start_minutes, end_minutes, '[')') with &&  
) where (status <> 'CANCELLED');
```

Impide que dos reservas del mismo negocio y día ocupen rangos horarios solapados; las canceladas quedan excluidas para permitir re-reservar el hueco. Detalle en el [Capítulo 5 §5.3](#).

A.2. Índices

Índice	Tabla	Columnas	Tipo
tenants_slug_idx	tenants	(slug)	Único
tenants_owner_idx	tenants	(owner_id)	Único
tenants_stripe_account_idx	tenants	(stripe_account_id)	Único parcial (WHERE NOT NULL)
services_tenant_idx	services	(tenant_id)	B-tree
schedules_tenant_idx	schedules	(tenant_id)	B-tree
customers_auth_user_idx	customers	(auth_user_id)	Único parcial (WHERE NOT NULL)
bookings_tenant_date_idx	bookings	(tenant_id, date)	B-tree
bookings_customer_idx	bookings	(customer_id)	B-tree
bookings_stripe_session_idx	bookings	(stripe_checkout_session_id)	Parcial (WHERE NOT NULL)
bookings_reminder_pending_idx	bookings	(date, status)	Parcial (WHERE status='CONFIRMED' AND reminder_sent_at IS NULL)

Anexo B. Políticas de seguridad a nivel de fila (RLS)

La seguridad a nivel de fila está **habilitada en las cinco tablas**. La siguiente tabla recoge todas las políticas vigentes (véase el análisis crítico en el [Capítulo 5 §5.7](#)).

Tabla	Política	Operación	Condición
tenants	Owner full access	ALL	auth.uid() = owner_id
tenants	Public read	SELECT	true
services	Owner full access	ALL	pertenencia al tenant del propietario
services	Public read active	SELECT	active = true
schedules	Owner full access	ALL	pertenencia al tenant del propietario
schedules	Public read	SELECT	true
customers	Public insert	INSERT	true ⚠
customers	Public read	SELECT	true ⚠
customers	Customer update own	UPDATE	auth_user_id = auth.uid()
bookings	Owner full access	ALL	pertenencia al tenant del propietario
bookings	Public insert	INSERT	true ⚠
bookings	Public read	SELECT	true ⚠
bookings	Customer update own bookings	UPDATE	propiedad vía customer_id ↔ auth_user_id

⚠ Las políticas marcadas son **deliberadamente permisivas** y constituyen la principal deuda de seguridad del sistema (habilitan el flujo de reserva anónimo a costa de un aislamiento multi-tenant incompleto). Su endurecimiento es la primera línea futura prioritaria ([Capítulo 7 §7.4.1](#)).

Anexo C. Historial de migraciones

#	Migración	Aportación
1	20260220221052_initial_schema.sql	Esquema inicial: 5 tablas, índices base y políticas RLS
2	20260221_add_stripe_checkout.sql	stripe_checkout_session_id en bookings + índice parcial
3	20260223_add_booking_policy.sql	Política de antelación en tenants : timezone , min_advance_minutes , max_advance_days
4	20260224_require_customer_phone.sql	phone obligatorio en customers (default ' ' , NOT NULL)
5	20260225_add_reminder_sent_at.sql	reminder_sent_at en bookings + índice de recordatorios pendientes
6	20260227_add_customer_portal.sql	auth_user_id y preferred_locale en customers ; RLS de autoservicio; índices
7	20260305_add_stripe_connect.sql	stripe_account_id / stripe_account_enabled en tenants + índice único parcial
8	20260306_add_tenant_profile.sql	Perfil de negocio y SEO en tenants (7 columnas)
9	20260422_prevent_booking_overlap.sql	Extensión btree_gist + restricción de exclusión bookings_no_overlap
10	20260629_add_onsite_payment.sql	allow_onsite_payment en tenants ; payment_method en bookings (CHECK ONLINE / ON_SITE) — habilita el pago presencial

Obsérvese que **ninguna migración añade una columna** *plan* : el modelo de planes (FREE/PRO) está diseñado en el dominio pero no persistido, por lo que todo negocio resuelve a *FREE* (véase [Capítulo 5 §5.5](#)).

Anexo D. Variables de entorno

Variables de configuración requeridas por el sistema (verificadas en el código fuente). Las prefijadas con `NEXT_PUBLIC_` se exponen al navegador; el resto son **secretos exclusivos del servidor**.

Variable	Ámbito	Propósito
NEXT_PUBLIC_SUPABASE_URL	Público	URL del proyecto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY	Público	Clave anónima de Supabase (sujeta a RLS)
SUPABASE_SERVICE_ROLE_KEY	Secreto	Clave de servicio privilegiada (<i>webhooks</i> , <i>cron</i>)
STRIPE_SECRET_KEY	Secreto	Clave secreta de la API de Stripe (crea cuentas Express, enlaces de <i>onboarding</i> y sesiones de pago)
STRIPE_WEBHOOK_SECRET	Secreto	Firma del <i>webhook</i> de plataforma (<code>account.updated</code>)
STRIPE_CONNECT_WEBHOOK_SECRET	Secreto	Firma del <i>webhook</i> de Connect (<code>checkout.session.completed</code>)
RESEND_API_KEY	Secreto	Clave de API de Resend
RESEND_FROM_DOMAIN	Secreto	Dominio remitente verificado de los correos (<code>reservas@...</code>)
SUPABASE_AUTH_HOOK_SECRET	Secreto	Firma del <i>Send Email Hook</i> de Supabase Auth (correos de autenticación i18n)
CRON_SECRET	Secreto	Autorización de la tarea programada de recordatorios
NEXT_PUBLIC_SITE_URL	Público	URL pública del sitio (SEO, <i>sitemap</i> , <i>robots</i>)
NEXT_PUBLIC_APP_URL	Público	URL base usada en las plantillas de correo

Anexo E. Evaluación de seguridad (OWASP Top 10:2021)

Autoevaluación del sistema frente al estándar **OWASP Top 10:2021** [17], verificada contra el código fuente. Estado:  cubierto ·  parcial ·  pendiente. Este anexo da soporte a las limitaciones de seguridad recogidas en el [Capítulo 7 §7.3](#) y a la línea futura de endurecimiento (§7.4.1).

Categoría	Estado	Valoración (evidencia y brecha)
A01 • Broken Access Control	🟡	✔️ Rutas privadas protegidas validando la sesión en servidor (<code>src/proxy.ts:28-51</code> , <code>auth.getUser()</code>); verificación de propiedad en cancelaciones (<code>admin/(dashboard)/bookings/actions.ts:15</code> ; <code>application/use-cases/cancel-customer-booking.ts:37</code>). ❌ Políticas RLS permisivas en <code>bookings / customers</code> (<code>supabase/migrations/20260220221052_initial_schema.sql:78-84</code>): el aislamiento entre <i>tenants</i> lo impone la aplicación, no la base de datos. 🟢 El <i>onboarding</i> de cuentas conectadas usa Connect Express (enlaces alojados por Stripe), por lo que no hay redireccionamiento OAuth con parámetro <code>state</code> que la plataforma deba validar.
A02 • Cryptographic Failures	✔️	HTTPS gestionado por Vercel; <i>hashing</i> de contraseñas y emisión de JWT delegados a Supabase Auth; secretos fuera del código (variables de entorno); firmas de <i>webhook</i> verificadas. (La política de contraseñas se evalúa en A07.)
A03 • Injection	🟡	✔️ Acceso a datos parametrizado mediante el cliente de Supabase (<code>.eq</code> , <code>.insert</code>), sin concatenación de SQL. ❌ Inyección HTML en correos: se interpolan datos del formulario público — <code>customer.name/email/phone</code> — sin escapar (<code>infrastructure/resend/email-templates.ts:42-44</code> ; también <code>tenantName / service.name</code> en <code>:12,:30</code>).
A04 • Insecure Design	🟡	✔️ Diseño defensivo notable: invariantes de dominio y restricción <code>EXCLUDE / btree_gist</code> contra solapamientos (Cap. 5 §5.3). ❌ Sin <i>rate limiting</i> en los puntos de entrada de la aplicación; sin modelo de amenazas formal.
A05 • Security Misconfiguration	🟡	❌ RLS mal configurada (véase A01). 🟡 No constan cabeceras de seguridad ni CSP (pendiente de verificación/adición en la configuración del <i>framework</i>).
A06 • Vulnerable & Outdated Components	🟡	✔️ Dependencias en versiones actuales (Next.js 16.1.6, React 19.2.3, etc.). ❌ Sin análisis automatizado de dependencias (<code>npm audit</code> / Dependabot) integrado aún en la CI.
A07 • Identification & Authentication Failures	🟡	✔️ Autenticación por Supabase Auth y Google OAuth, verificación de email obligatoria en el registro (confirmación por enlace, <code>api/auth/confirm</code>), sesión validada en servidor, recuperación de contraseña funcional (<code>admin/reset-password</code>). 🟡 política de contraseñas débil (mínimo 6, sin requisitos de complejidad; <code>supabase/config.toml:175,178</code>); MFA deshabilitado.
A08 • Software & Data Integrity Failures	✔️	✔️ Verificación de firma en ambos webhooks (<code>api/webhooks/stripe-connect/route.ts:23</code> ; <code>api/webhooks/stripe/route.ts:21</code>) y guarda de idempotencia (<code>status === PENDING</code>). 🟡 Sin <i>idempotency keys</i> de Stripe en las llamadas salientes.
A09 • Security Logging & Monitoring Failures	❌	Registro limitado a <code>console.error</code> ; sin trazas estructuradas, alertas ni auditoría.
A10 • Server-Side Request Forgery (SSRF)	✔️	Sin peticiones de servidor a URLs controladas por el usuario; las salidas se dirigen a proveedores fijos (Stripe, Supabase, Resend). Exposición baja.

E.1. Síntesis y prioridades

El sistema es **sólido** en A02, A08 y A10, y **parcial** —con brechas concretas y localizadas— en A01, A03, A04, A05, A06, A07 y A09. Las tres intervenciones de mayor retorno, ya recogidas como trabajo futuro en el [Capítulo 7 §7.4.1](#), son: (1) endurecer las políticas **RLS** (A01/A05); (2) **escapar el HTML** de los correos (A03); y (3) incorporar **logging estructurado** (A09). Su resolución elevaría la mayoría de categorías a  y sustentaría un capítulo de modelado de amenazas.

Anexo F. Estrategia de pruebas y política de cobertura

Complementa el [Capítulo 6](#) detallando la **naturaleza de cada tipo de prueba** y la **política de cobertura diferenciada por capa** —el principio de que cada tipo de prueba justifica un objetivo de cobertura distinto—.

F.1. Tipos de prueba y objetivo de cobertura

Tipo	Naturaleza	Casos	¿En cobertura?	Objetivo recomendado
Unitarias de dominio (objetos de valor y servicios puros)	Aisladas, deterministas, sin E/S	123 (51 %)	Sí	~100 % líneas / ~95 % ramas
De casos de uso (aplicación, con <i>test doubles</i> en memoria)	"Sociables": verifican la orquestación con dobles, no la implementación	57 (24 %)	Sí	~90 % líneas / ~85 % ramas
De adaptador (infraestructura, con <i>mocks</i> de los SDK)	Verifican el <i>mapeo</i> (p. ej. <code>23P01</code> → <code>SlotTakenError</code> , cálculo de comisión), no la dependencia real	60 (25 %)	Sí	~85 % líneas / ~75 % ramas
Integración real (contra PostgreSQL efímero)	Ejercitaría la restricción <code>EXCLUDE</code> y las políticas RLS de extremo a extremo	0	—	flujos críticos (línea futura)
E2E (Playwright automatizado)	Recorrido completo del usuario contra el despliegue	2 (automatizadas)	No (valida presentación)	flujo reservar → confirmar

F.2. Configuración actual

La cobertura se mide sobre las capas con lógica —dominio, aplicación e infraestructura— y define un **umbral** que actúa de puerta de calidad (`vitest.config.ts`):

```
coverage: {
  provider: 'v8',
  include: ['src/domain/**', 'src/application/**', 'src/infrastructure/**'],
  exclude: [/ * tipos e interfaces, clientes de SDK y glue de sesión */],
  thresholds: { statements: 85, functions: 85, lines: 85, branches: 75 },
}
```

Se **excluye** el *glue* sin lógica (clientes de SDK, `server.ts` , helpers de sesión) y la **capa de presentación** (`src/app`), cuya validación por líneas sería engañosa —esta se cubre con las **pruebas E2E de Playwright**—. La medición se ejecuta en **integración continua** (`.github/workflows/ci.yml`), que **falla el *build*** por debajo del umbral. La única categoría aún pendiente es la de **integración real** contra un PostgreSQL efímero.

F.3. Política propuesta

El paso natural, coherente con el objetivo de calidad de la tesis, es declarar **umbrales diferenciados por capa** y ejecutarlos en CI:

```
coverage: {
  provider: 'v8',
  include: ['src/domain/**', 'src/application/**'],
  thresholds: {
    'src/domain/**': { lines: 100, branches: 95 },
    'src/application/**': { lines: 90, branches: 85 },
  },
}
```

Complementado con una capa de **pruebas de integración** (la restricción `EXCLUDE` y la RLS contra un PostgreSQL real —p. ej. Supabase local o *Testcontainers*—) y **pruebas E2E** (Playwright del flujo de reserva), que cubren la infraestructura y la presentación que la cobertura unitaria, por diseño, no contempla.